

ĐẠI HỌC QUỐC GIA THÀNH PHỐ HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC CÔNG NGHỆ THÔNG TIN
KHOA MẠNG MÁY TÍNH VÀ TRUYỀN THÔNG



LÊ XUÂN HOÀNG
ĐẶNG MINH DZŨ

ĐỒ ÁN CHUYÊN NGÀNH

Thiết kế và triển khai hệ thống quản lý tài nguyên Cloud sử dụng AI
Agent trên nền tảng OpenNebula

Design and Implement an AI Agent-based Cloud Resource Management System on
the OpenNebula Platform

CỬ NHÂN NGÀNH MẠNG MÁY TÍNH VÀ TRUYỀN THÔNG DỮ LIỆU

TP. HỒ CHÍ MINH, 2026

**ĐẠI HỌC QUỐC GIA THÀNH PHỐ HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC CÔNG NGHỆ THÔNG TIN
KHOA MẠNG MÁY TÍNH VÀ TRUYỀN THÔNG**



**LÊ XUÂN HOÀNG – 23520524
ĐẶNG MINH DZŨ – 23520404**

ĐỒ ÁN CHUYÊN NGÀNH

**Thiết kế và triển khai hệ thống quản lý tài nguyên Cloud sử dụng AI
Agent trên nền tảng OpenNebula**

**Design and Implement an AI Agent-based Cloud Resource Management System on
the OpenNebula Platform**

CỬ NHÂN NGÀNH MẠNG MÁY TÍNH VÀ TRUYỀN THÔNG DỮ LIỆU

**GIẢNG VIÊN HƯỚNG DẪN
ThS. Trần Thị Dung**

TP. HỒ CHÍ MINH, 2026

THÔNG TIN HỘI ĐỒNG BẢO VỆ KHÓA LUẬN

Hội đồng chấm khóa luận tốt nghiệp, thành lập theo Quyết định số ngày ..
..... của Hiệu trưởng Trường Đại học Công nghệ Thông tin.

1. Chủ tịch:

2. Thư ký:

3. Ủy viên:

LỜI CẢM ƠN

Chúng em xin trân trọng gửi lời cảm ơn sâu sắc đến tất cả những người đã đồng hành, hỗ trợ và tạo điều kiện thuận lợi để chúng em hoàn thành đồ án chuyên ngành này.

Trước hết, chúng em xin bày tỏ lòng biết ơn chân thành đến ThS. Trần Thị Dung – giảng viên hướng dẫn. Cô đã luôn tận tâm chỉ dẫn, định hướng và góp ý quý báu trong suốt quá trình thực hiện. Những nhận xét sâu sắc và kịp thời của cô đã giúp chúng em hoàn thiện nội dung, cũng như tiếp cận vấn đề một cách khoa học và hệ thống hơn.

Chúng em cũng xin chân thành cảm ơn Ban Giám hiệu Trường Đại học Công nghệ Thông tin, Đại học Quốc gia Thành phố Hồ Chí Minh, cùng toàn thể quý thầy cô Khoa Mạng Máy tính và Truyền thông đã trang bị cho chúng em nền tảng kiến thức vững chắc trong suốt quá trình học tập. Đây chính là cơ sở quan trọng giúp chúng em có thể triển khai và hoàn thành đồ án.

Mặc dù đã nỗ lực hết mình, đồ án vẫn khó tránh khỏi những hạn chế nhất định. Chúng em rất mong nhận được những ý kiến đóng góp quý báu từ quý thầy cô và hội đồng để có thể tiếp tục hoàn thiện hơn trong thời gian tới.

Nhóm tác giả

TÓM TẮT

Đề tài tập trung thiết kế và triển khai một hệ thống quản lý tài nguyên Cloud quy mô nhỏ dựa trên nền tảng OpenNebula, hướng đến việc tối ưu hóa việc sử dụng tài nguyên hạ tầng như CPU, RAM và mạng trong môi trường IaaS. Hệ thống được xây dựng với kiến trúc gồm nhiều thành phần như OpenNebula để quản lý và phân bổ máy ảo, hệ thống giám sát sử dụng Prometheus để thu thập dữ liệu và Grafana để trực quan hóa thông tin, giúp người quản trị theo dõi trạng thái hệ thống một cách hiệu quả và trực quan.

Dữ liệu giám sát đóng vai trò quan trọng trong việc cung cấp thông tin đầu vào cho AI Agent nhằm phân tích tình trạng hoạt động của hệ thống. AI Agent trong đề tài được triển khai theo hướng kết hợp MCP Server và framework CrewAI, đồng thời sử dụng API của Groq để nâng cao khả năng suy luận, xử lý ngữ cảnh và phối hợp giữa các agent. Nhờ đó, hệ thống có thể tự động đưa ra các đề xuất như cân bằng tải giữa các host, phát hiện và cảnh báo tình trạng quá tải, cũng như hỗ trợ quyết định mở rộng hoặc thu hẹp tài nguyên một cách linh hoạt.

Bên cạnh đó, phương pháp GitOps được áp dụng nhằm quản lý cấu hình hệ thống thông qua Git, giúp đảm bảo tính nhất quán, khả năng kiểm soát phiên bản và dễ dàng theo dõi các thay đổi trong quá trình vận hành. Giao diện Web được xây dựng để tích hợp toàn bộ chức năng giám sát và phân tích, cho phép người quản trị quan sát dữ liệu theo thời gian thực và tiếp nhận các đề xuất từ AI Agent một cách thuận tiện.

Ngoài ra, hệ thống còn được thiết kế với khả năng mở rộng trong tương lai, cho phép tích hợp thêm các cơ chế tự động hóa nâng cao như tự động scale tài nguyên hoặc tối ưu chi phí vận hành. Việc kết hợp giữa Cloud Computing và AI Agent không chỉ giúp nâng cao hiệu quả sử dụng tài nguyên mà còn giảm thiểu sự can thiệp thủ công của con người trong quá trình quản lý hệ thống.

Kết quả của đề tài hướng đến việc xây dựng một hệ thống Cloud thông minh có khả năng giám sát, phân tích và hỗ trợ ra quyết định một cách tự động, tạo nền tảng cho các nghiên cứu tiếp theo về hệ thống tự quản và hạ tầng Cloud tích hợp AI trong thực tế.

MỤC LỤC

Thông tin hội đồng bảo vệ khóa luận	i
Lời cảm ơn	ii
Tóm tắt	iii
Mục lục	iv
Danh mục các bảng	vii
Danh mục các hình vẽ và đồ thị	viii
Danh mục từ viết tắt	ix
Chương 1. Mở đầu	1
1.1 Lý do chọn đề tài	1
1.2 Các nghiên cứu liên quan	1
1.3 Mục tiêu, đối tượng và phạm vi nghiên cứu	2
1.3.1 Mục tiêu nghiên cứu	2
1.3.2 Đối tượng nghiên cứu	2
1.3.3 Phạm vi nghiên cứu	2
1.4 Phương pháp nghiên cứu	3
1.5 Các đóng góp chính của đề tài	3
1.6 Cấu trúc Đồ án chuyên ngành	4
Chương 2. Cơ sở lý thuyết	5
2.1 Tổng quan về môi trường Cloud	5
2.1.1 Khái niệm và mô hình Cloud Computing	5
2.1.2 Nền tảng OpenNebula	5
2.2 Giám sát tài nguyên hệ thống	6
2.2.1 Prometheus	6
2.2.2 Grafana	6
2.3 AI Agent trong quản lý tài nguyên Cloud	7
2.3.1 Tổng quan về AI Agent	7

MỤC LỤC

2.3.2	Groq API	7
2.3.3	CrewAI Framework	7
2.3.4	MCP Server	8
2.3.5	Sự kết hợp giữa Groq API, CrewAI và MCP Server	8
2.3.6	Tổng quan về MAPE-K	9
2.4	Web Application	10
2.4.1	Tổng quan	10
2.4.2	Frontend	10
2.4.3	Backend	10
2.5	GitOps trong quản lý cấu hình hệ thống	10
2.5.1	Tổng quan về GitOps	10
2.5.2	Ứng dụng GitOps trong đề tài	10
Chương 3.	Phương pháp thực hiện	12
3.1	Mô hình triển khai hệ thống	12
3.2	Phương pháp triển khai hệ thống	12
3.2.1	Kiến trúc triển khai tổng thể	12
3.2.2	Quy trình triển khai	13
3.3	Phương pháp giám sát hệ thống	13
3.3.1	Thu thập dữ liệu	13
3.3.2	Các chỉ số giám sát về ngưỡng cảnh báo	14
3.3.3	Phương pháp trực quan hóa với Grafana	14
3.4	Phương pháp tích hợp AI theo kiến trúc MAPE-K	14
3.4.1	Ánh xạ MAPE-K vào các thành phần hệ thống	14
3.4.2	Luồng hoạt động chi tiết	15
3.4.3	Chế độ hoạt động của AI Agent	16
3.5	Phương pháp thiết kế và xây dựng Web App	16
3.5.1	Kiến trúc tổng thể Web App	16
3.5.2	Phương pháp thiết kế giao diện Frontend	17
3.5.3	Phương pháp xây dựng Backend API	18
3.6	Phương pháp quản lý cấu hình bằng GitOps	18
3.7	Phương pháp kiểm tra và đánh giá hệ thống	19
Chương 4.	Triển khai hệ thống	21

MỤC LỤC

4.1	Môi trường và mô hình triển khai	21
4.2	Triển khai nền tảng Cloud OpenNebula	21
4.2.1	Thiết lập kho lưu trữ và cập nhật hệ thống	21
4.2.2	Triển khai nút quản trị (Front-end Node)	22
4.2.3	Triển khai nút tính toán KVM (Host Node) và cấu hình xác thực	23
4.3	Triển khai kiến trúc giám sát	24
4.3.1	Cài đặt tác nhân thu thập dữ liệu (Node Exporter)	24
4.3.2	Cấu hình máy chủ Prometheus trung tâm	24
4.3.3	Triển khai Grafana Dashboard	25
4.4	Triển khai AI Agent theo kiến trúc MAPE-K	26
4.4.1	Giai đoạn Monitor & Analyze: Thu thập và Đánh giá chỉ số	27
4.4.2	Giai đoạn Plan: Khởi tạo AI Agent	28
4.4.3	Kỹ thuật Adapter: Giải quyết xung đột Schema	29
4.4.4	Giai đoạn Execute: Phê duyệt và Thực thi tự động	29
4.4.5	Giai đoạn Knowledge: Lưu trữ tri thức và cập nhật ngữ cảnh	31
4.5	Thiết kế và triển khai ứng dụng Web quản trị	32
4.5.1	Kiến trúc giao diện và lựa chọn công nghệ Front-end	33
4.5.2	Phân hệ giám sát và kỹ thuật đồng bộ dữ liệu	34
4.5.3	Xử lý Logic Chatbox và hiển thị đề xuất AI	34
4.6	Tích hợp CrewAI và giao thức MCP	36
4.6.1	Điều phối kịch bản tự động hóa với CrewAI	36
4.6.2	Đóng gói công cụ hạ tầng qua FastMCP Server	36
4.6.3	Cơ chế giao tiếp khép kín giữa LLM và Hệ thống	37
4.7	Kiểm thử và đánh giá kết quả hệ thống	38
4.7.1	Kịch bản kiểm thử	39
4.7.2	Đánh giá kết quả	42
Chương 5.	Kết luận và hướng phát triển	43
5.1	Kết quả đạt được	43
5.2	Hạn chế của đề tài	45
5.3	Hướng phát triển	45
Phụ lục A.	Công bố khoa học	47
Tài liệu tham khảo		48

DANH MỤC CÁC BẢNG

3.1	Các chỉ số giám sát và ngưỡng cảnh báo của hệ thống	14
3.2	Mô hình hoạt động MAPE-K trong hệ thống	14
3.3	Cấu trúc giao diện hệ thống	17
3.4	Các kịch bản kiểm thử hệ thống	19
3.5	Các chỉ tiêu đánh giá hệ thống	19
4.1	Quy hoạch tài nguyên và địa chỉ IP hệ thống	21
4.2	Các thư viện chính và chức năng trong hệ thống	32

DANH MỤC CÁC HÌNH VẼ VÀ ĐỒ THỊ

3.1	Mô hình kiến trúc tổng thể của hệ thống	12
4.1	Giao diện của OpenNebula Frontend	23
4.2	Giao diện của Prometheus	25
4.3	Xây dựng hệ thống thu thập dữ liệu từ Prometheus	25
4.4	Giao diện của Grafana	26
4.5	Kiến trúc MAPE-K	27
4.6	Chu trình MAPE-K trong hệ thống giám sát và quản lý tài nguyên Cloud . .	32
4.7	Mô hình database bảng AI_LOGS	32
4.8	Giao diện Web của trang Dashboard	35
4.9	Giao diện Web của trang AI Agent	35
4.10	Giao diện Web của trang System Log & AI Chatbox	35
4.11	Kiểm tra chức năng tạo các VM	38
4.12	Các thông số của hệ thống	38
4.13	Chạy kịch bản stress-ng trên Host Node	40
4.14	Dashboard hệ thống báo critical khi chạy kịch bản	40
4.15	Chat với AI để tìm process và yêu cầu kill process	40
4.16	Kết quả sau khi kill process	41
5.1	Tổng kết kết quả đạt được theo mô hình MAPE-K	44

DANH MỤC TỪ VIẾT TẮT

AI	Artificial Intelligence
API	Application Programming Interface
CPU	Central Processing Unit
LLM	Large Language Model
MCP	Model Context Protocol
RAM	Random Access Memory

Chương 1. MỞ ĐẦU

1.1 Lý do chọn đề tài

Trong xu hướng phát triển của điện toán đám mây, việc quản lý và tối ưu tài nguyên hạ tầng (CPU, RAM, mạng) trong môi trường IaaS ngày càng trở nên quan trọng nhưng vẫn còn nhiều thách thức. Đồng thời, sự phát triển của trí tuệ nhân tạo cho phép tự động hóa quá trình phân tích và hỗ trợ ra quyết định trong quản lý hệ thống.

Việc kết hợp các công nghệ như OpenNebula, Prometheus, Grafana với AI Agent (sử dụng MCP Server, CrewAI và API Groq) giúp xây dựng một hệ thống giám sát và quản lý tài nguyên thông minh, có khả năng phân tích và đề xuất điều chỉnh phù hợp.

Vì vậy, đề tài được lựa chọn nhằm nghiên cứu và triển khai một mô hình Cloud tích hợp AI, góp phần nâng cao hiệu quả quản lý tài nguyên và làm nền tảng cho các hệ thống tự động trong tương lai.

1.2 Các nghiên cứu liên quan

Bài toán quản lý và tối ưu tài nguyên trong môi trường Cloud đã được nghiên cứu từ nhiều góc độ khác nhau. Một số hướng tiếp cận tiêu biểu bao gồm:

Phương pháp lập lịch và phân bổ tài nguyên truyền thống: Các nền tảng Cloud như OpenNebula, OpenStack sử dụng các thuật toán lập lịch dựa trên quy tắc hoặc heuristic để phân bổ tài nguyên cho máy ảo. Mặc dù đảm bảo tính ổn định và dễ triển khai, các phương pháp này còn hạn chế trong việc thích nghi với sự thay đổi động của tải hệ thống và chưa tận dụng được dữ liệu lịch sử.

Phương pháp giám sát và cảnh báo: Các hệ thống như Prometheus kết hợp Grafana cho phép thu thập và trực quan hóa dữ liệu tài nguyên theo thời gian thực, đồng thời thiết lập các ngưỡng cảnh báo. Tuy nhiên, các giải pháp này chủ yếu mang tính phản ứng, phụ thuộc vào cấu hình thủ công và chưa hỗ trợ phân tích sâu hay đề xuất quyết định tự động.

Ứng dụng học máy và AI trong quản lý Cloud: Một số nghiên cứu gần đây áp dụng các mô hình học máy để dự đoán tải hệ thống, tối ưu phân bổ tài nguyên hoặc phát hiện bất thường. Mặc dù cải thiện khả năng tự động hóa, các phương pháp này thường yêu cầu dữ liệu lớn, khó triển khai thực tế và thiếu khả năng giải thích rõ ràng cho người quản trị.

Kiến trúc AI Agent và đa tác nhân: Các framework như CrewAI cho phép xây dựng hệ thống đa tác nhân, trong đó mỗi tác nhân đảm nhiệm một vai trò cụ thể như thu thập dữ

liệu, phân tích và đề xuất. Khi kết hợp với Gemini API, hệ thống có thể tận dụng khả năng suy luận mạnh mẽ của LLM để nâng cao chất lượng phân tích và đưa ra quyết định chính xác hơn. Cách tiếp cận này giúp cải thiện tính linh hoạt, khả năng mở rộng và hiệu quả tổng thể của hệ thống so với mô hình đơn tác nhân.

Đồ án này kế thừa các hướng tiếp cận trên, đồng thời đề xuất một mô hình tích hợp giữa hệ thống Cloud, giám sát tài nguyên và AI Agent nhằm xây dựng một hệ thống quản lý tài nguyên thông minh, có khả năng phân tích và hỗ trợ ra quyết định hiệu quả trong môi trường thực tế.

1.3 Mục tiêu, đối tượng và phạm vi nghiên cứu

1.3.1 Mục tiêu nghiên cứu

Mục tiêu của đề tài là thiết kế và triển khai một hệ thống quản lý tài nguyên Cloud trên nền tảng OpenNebula, tích hợp AI Agent nhằm hỗ trợ giám sát, phân tích và tối ưu việc sử dụng tài nguyên. Cụ thể, đề tài hướng đến việc xây dựng môi trường Cloud cơ bản, thu thập và trực quan hóa dữ liệu tài nguyên hệ thống, đồng thời ứng dụng AI Agent để phân tích dữ liệu và đưa ra các đề xuất như cân bằng tải, cảnh báo quá tải và điều chỉnh tài nguyên.

Ngoài ra, đề tài cũng nhằm phát triển giao diện Web phục vụ quản trị hệ thống và áp dụng GitOps trong quản lý cấu hình, từ đó nâng cao tính tự động hóa, khả năng quản lý và mở rộng hệ thống trong tương lai.

1.3.2 Đối tượng nghiên cứu

Đối tượng nghiên cứu của đề tài là hệ thống quản lý tài nguyên trong môi trường Cloud Computing theo mô hình IaaS, bao gồm các thành phần như hạ tầng ảo hóa, máy ảo, tài nguyên CPU, RAM, lưu trữ và mạng. Bên cạnh đó, đề tài tập trung nghiên cứu các công cụ giám sát hệ thống và phương pháp quản lý cấu hình theo GitOps.

Đề tài cũng nghiên cứu việc ứng dụng AI Agent trong phân tích dữ liệu giám sát và hỗ trợ ra quyết định quản lý tài nguyên, cụ thể thông qua việc kết hợp MCP Server, CrewAI và Groq API trong môi trường Cloud thực tế.

1.3.3 Phạm vi nghiên cứu

Đề tài tập trung nghiên cứu và triển khai hệ thống quản lý tài nguyên Cloud ở quy mô nhỏ trong môi trường thử nghiệm, sử dụng nền tảng OpenNebula trên hệ điều hành Ubuntu

Server. Hệ thống chủ yếu quản lý các tài nguyên cơ bản như CPU, RAM, lưu trữ và mạng của các máy ảo.

Phạm vi đề tài giới hạn ở việc giám sát và phân tích dữ liệu thông qua Prometheus và Grafana, đồng thời ứng dụng AI Agent để đưa ra các đề xuất hỗ trợ quản lý tài nguyên, chưa thực hiện tự động điều khiển hoàn toàn hệ thống.

Đề tài chỉ triển khai các chức năng cơ bản của Web App phục vụ quản trị và áp dụng GitOps trong quản lý cấu hình, chưa mở rộng sang các hệ thống Cloud quy mô lớn hoặc môi trường production thực tế.

1.4 Phương pháp nghiên cứu

Nhóm áp dụng kết hợp các phương pháp sau: **(1) Nghiên cứu tài liệu** — khảo sát các công trình liên quan về Cloud Computing, quản lý tài nguyên trong môi trường IaaS, hệ thống giám sát và ứng dụng AI Agent; **(2) Thiết kế và hiện thực hóa hệ thống** — xây dựng hệ thống theo quy trình Agile, chia thành các sprint để triển khai từng thành phần như OpenNebula, hệ thống giám sát, AI Agent và Web App; **(3) Thực nghiệm đánh giá** — triển khai hệ thống trên môi trường Ubuntu Server và thu thập dữ liệu tài nguyên thông qua Prometheus để kiểm thử hoạt động; **(4) Phân tích và đánh giá** — so sánh hiệu quả quản lý tài nguyên trước và sau khi áp dụng AI Agent, đồng thời đánh giá chất lượng các đề xuất do hệ thống đưa ra dựa trên các kịch bản thực tế.

1.5 Các đóng góp chính của đề tài

Đề tài cung cấp một giải pháp quản lý tài nguyên Cloud có khả năng giám sát và hỗ trợ ra quyết định thông minh, giúp doanh nghiệp tối ưu việc sử dụng hạ tầng, giảm chi phí vận hành và nâng cao hiệu quả khai thác tài nguyên.

Việc tích hợp AI Agent cho phép phân tích dữ liệu hệ thống và đưa ra các đề xuất tự động như cân bằng tải, cảnh báo quá tải, từ đó giảm phụ thuộc vào quản trị thủ công và hạn chế rủi ro trong vận hành.

Ngoài ra, mô hình hệ thống được xây dựng có tính mở và dễ mở rộng, tạo nền tảng để phát triển các hệ thống Cloud tự động (auto-scaling, self-healing) trong tương lai, phù hợp với xu hướng ứng dụng AI trong quản lý hạ tầng.

1.6 Cấu trúc Đồ án chuyên ngành

Đồ án với đề tài “Thiết kế và triển khai hệ thống quản lý tài nguyên Cloud sử dụng AI Agent trên nền tảng OpenNebula” được trình bày bao gồm 5 chương. Nội dung tóm tắt từng chương được trình bày như sau:

- **Chương 1: Mở đầu.** Trình bày lý do chọn đề tài, các nghiên cứu liên quan, mục tiêu, phạm vi nghiên cứu và cấu trúc đồ án.
- **Chương 2: Cơ sở lý thuyết.** Giới thiệu các nền tảng lý thuyết về môi trường Cloud, framework CrewAI, MCP Server, công cụ giám sát và trực quan hóa hệ thống mã nguồn mở và các công nghệ xây dựng hệ thống.
- **Chương 3: Phương pháp thực hiện.** Trình bày chi tiết kiến trúc hệ thống, thiết kế các thành phần trong pipeline và các quyết định kỹ thuật chính được sử dụng trong quá trình xây dựng hệ thống quản lý tài nguyên Cloud tích hợp AI Agent.
- **Chương 4: Triển khai hệ thống.** Trình bày kết quả triển khai, kiểm thử hệ thống và phân tích kết quả thực nghiệm.
- **Chương 5: Kết luận và hướng phát triển.** Tóm tắt những kết quả đạt được, hạn chế và định hướng phát triển tương lai.

Chương 2. CƠ SỞ LÝ THUYẾT

Chương này trình bày phương pháp thực hiện đề tài thông qua việc mô tả kiến trúc tổng thể của hệ thống quản lý tài nguyên Cloud tích hợp AI Agent. Nội dung tập trung vào cách tổ chức các thành phần chính như nền tảng Cloud, hệ thống giám sát, AI Agent và giao diện Web. Đồng thời, chương cũng phân tích thiết kế các tác nhân trong pipeline xử lý dữ liệu và các quyết định kỹ thuật quan trọng trong quá trình triển khai. Qua đó, làm rõ cách hệ thống được xây dựng nhằm đảm bảo tính hiệu quả, khả năng mở rộng và phù hợp với mục tiêu đề tài.

2.1 Tổng quan về môi trường Cloud

2.1.1 Khái niệm và mô hình Cloud Computing

Cloud Computing là mô hình cung cấp tài nguyên tính toán như máy chủ, lưu trữ và mạng thông qua Internet, cho phép người dùng sử dụng tài nguyên linh hoạt theo nhu cầu mà không cần đầu tư trực tiếp vào hạ tầng vật lý.

Ba mô hình dịch vụ chính của Cloud Computing gồm IaaS, PaaS và SaaS. Trong đó, IaaS cung cấp quyền quản lý các tài nguyên hạ tầng như máy ảo, CPU, RAM và mạng, phù hợp cho việc triển khai và nghiên cứu hệ thống Cloud.

Do môi trường Cloud có tính động và quy mô lớn, việc quản lý tài nguyên thủ công gặp nhiều hạn chế. Vì vậy, các giải pháp tự động hóa và ứng dụng AI ngày càng được quan tâm nhằm nâng cao hiệu quả vận hành và tối ưu hóa việc sử dụng tài nguyên.

2.1.2 Nền tảng OpenNebula

OpenNebula là nền tảng mã nguồn mở dùng để xây dựng và quản lý hạ tầng Cloud theo mô hình IaaS. Hệ thống cho phép triển khai, quản lý và giám sát các máy ảo trên nhiều máy chủ vật lý, đồng thời hỗ trợ phân bổ tài nguyên hiệu quả.

Kiến trúc OpenNebula gồm máy chủ quản lý trung tâm (Frontend) và các Host Node thực thi máy ảo. Nền tảng cung cấp các thành phần quan trọng như bộ lập lịch, quản lý mạng, lưu trữ và giao diện quản trị Sunstone.

Nhờ khả năng triển khai đơn giản và dễ tích hợp với các hệ thống giám sát bên ngoài, OpenNebula là môi trường phù hợp để xây dựng và thử nghiệm các giải pháp quản lý tài nguyên thông minh ứng dụng AI Agent.

2.2 Giám sát tài nguyên hệ thống

2.2.1 Prometheus

Prometheus là hệ thống giám sát và cảnh báo mã nguồn mở, được sử dụng rộng rãi trong các môi trường Cloud và Cloud Native. Hệ thống thu thập dữ liệu theo mô hình pull-based và lưu trữ dưới dạng chuỗi thời gian (time-series).

Các thành phần chính của Prometheus gồm:

- **Prometheus Server:** Thu thập, lưu trữ và truy vấn dữ liệu giám sát.
- **Exporter:** Thu thập các chỉ số từ hệ thống và cung cấp cho Prometheus, điển hình là Node Exporter.
- **Alertmanager:** Quản lý và gửi cảnh báo khi phát hiện các sự kiện bất thường.

Trong đề tài, Node Exporter được triển khai trên các Host Node của OpenNebula để thu thập dữ liệu CPU, RAM, lưu trữ và mạng. Dữ liệu này được Prometheus lưu trữ và cung cấp cho AI Agent thực hiện phân tích tài nguyên.

2.2.2 Grafana

Grafana là nền tảng trực quan hóa dữ liệu mã nguồn mở, cho phép hiển thị thông tin giám sát dưới dạng dashboard và biểu đồ theo thời gian thực. Grafana thường được sử dụng cùng Prometheus để theo dõi trạng thái hệ thống một cách trực quan.

Các tính năng nổi bật của Grafana gồm:

- **Dashboard linh hoạt:** Hỗ trợ nhiều loại biểu đồ và tùy chỉnh giao diện theo nhu cầu giám sát.
- **Đa nguồn dữ liệu:** Kết nối với nhiều hệ thống như Prometheus, InfluxDB và MySQL.
- **Cảnh báo tích hợp:** Hỗ trợ thiết lập ngưỡng và gửi thông báo khi phát hiện bất thường.

Trong đề tài, Grafana được sử dụng để trực quan hóa các chỉ số tài nguyên từ Prometheus như CPU, RAM, lưu trữ và mạng, giúp quản trị viên theo dõi tình trạng hạ tầng Cloud hiệu quả.

2.3 AI Agent trong quản lý tài nguyên Cloud

2.3.1 Tổng quan về AI Agent

AI Agent là hệ thống phần mềm thông minh có khả năng quan sát môi trường, suy luận và thực hiện hành động nhằm đạt được mục tiêu được xác định trước. Khác với chatbot truyền thống chỉ phản hồi yêu cầu, AI Agent có thể hoạt động theo vòng lặp liên tục: Quan sát → Suy luận → Hành động.

Trong quản lý hạ tầng Cloud, AI Agent có thể phân tích dữ liệu giám sát, phát hiện các dấu hiệu bất thường về tài nguyên và đưa ra các đề xuất tối ưu hóa hệ thống, giúp giảm tải công việc cho quản trị viên.

Một AI Agent hiện đại thường bao gồm ba thành phần chính: mô hình ngôn ngữ lớn để suy luận, các công cụ để tương tác với hệ thống bên ngoài và cơ chế điều phối để quản lý luồng thực thi giữa các tác vụ hoặc nhiều Agent.

2.3.2 Groq API

Groq API là nền tảng cung cấp khả năng suy luận cho các mô hình ngôn ngữ lớn với tốc độ xử lý cao và độ trễ thấp. Nền tảng hỗ trợ nhiều mô hình mã nguồn mở như Llama, Gemma và Qwen, đồng thời cung cấp giao diện API tương thích với OpenAI, giúp dễ dàng tích hợp vào các hệ thống AI Agent.

Tính năng quan trọng cho AI Agent:

- **High-Speed Inference:** Tốc độ phản hồi nhanh, hỗ trợ AI Agent phân tích và đưa ra quyết định gần như theo thời gian thực.
- **Tool Calling:** Hỗ trợ gọi các công cụ và dịch vụ bên ngoài, cho phép Agent tương tác với hệ thống giám sát và quản lý hạ tầng.
- **OpenAI-Compatible API:** Tương thích với chuẩn API của OpenAI, giúp đơn giản hóa quá trình phát triển và tích hợp ứng dụng.
- **Model Flexibility:** Hỗ trợ nhiều mô hình ngôn ngữ khác nhau, cho phép lựa chọn mô hình phù hợp với yêu cầu về hiệu năng và chi phí.

2.3.3 CrewAI Framework

CrewAI là framework mã nguồn mở được phát triển bằng Python, hỗ trợ xây dựng và điều phối các hệ thống đa Agent. Framework cho phép nhiều Agent với các vai trò khác nhau

CHƯƠNG 2. CƠ SỞ LÝ THUYẾT

phối hợp để thực hiện các nhiệm vụ phức tạp một cách có tổ chức.

Các thành phần chính của CrewAI gồm:

- **Agent:** Thực thể thực hiện nhiệm vụ, được định nghĩa bởi vai trò và mục tiêu cụ thể.
- **Task:** Công việc được giao cho Agent cùng với đầu ra mong muốn.
- **Crew:** Tập hợp các Agent và Task cùng phối hợp để hoàn thành mục tiêu chung.
- **Process:** Cơ chế điều phối luồng thực thi giữa các Agent, có thể theo tuần tự hoặc phân cấp.
- **Tool:** Các công cụ cho phép Agent tương tác với hệ thống bên ngoài như API hoặc cơ sở dữ liệu.

Trong đề tài, CrewAI được sử dụng để điều phối các Agent thực hiện thu thập dữ liệu, phân tích tài nguyên và đưa ra các đề xuất quản lý hạ tầng Cloud.

2.3.4 MCP Server

Model Context Protocol là giao thức mã nguồn mở giúp kết nối AI Agent với các nguồn dữ liệu và công cụ bên ngoài thông qua một chuẩn giao tiếp thống nhất. MCP đóng vai trò cầu nối giữa mô hình AI và các hệ thống thực tế, giúp việc tích hợp và mở rộng trở nên đơn giản hơn.

Các thành phần chính của MCP gồm:

- **MCP Server:** Cung cấp các công cụ và dữ liệu cho AI Agent.
- **MCP Client:** Thành phần phía AI Agent dùng để gửi yêu cầu đến MCP Server.
- **Tool Discovery:** Cho phép Agent tự động khám phá và sử dụng các công cụ có sẵn.

Trong đề tài, MCP Server được sử dụng để cung cấp các công cụ truy vấn dữ liệu từ Prometheus và OpenNebula. Thông qua MCP, AI Agent có thể thu thập dữ liệu giám sát, trạng thái máy ảo và tài nguyên hạ tầng mà không cần tích hợp trực tiếp với từng hệ thống, giúp tăng tính linh hoạt và khả năng mở rộng.

2.3.5 Sự kết hợp giữa Groq API, CrewAI và MCP Server

Ba công nghệ được tích hợp để xây dựng hệ thống AI Agent hoàn chỉnh:

- **Groq API** cung cấp khả năng suy luận nhanh cho các Agent, hỗ trợ phân tích dữ liệu và đưa ra phản hồi với độ trễ thấp.
- **CrewAI Framework** điều phối các Agent chuyên biệt, tổ chức quy trình làm việc từ thu thập dữ liệu, phân tích đến đưa ra đề xuất.
- **MCP Server** đóng vai trò cầu nối, cho phép Agent truy cập dữ liệu và dịch vụ từ Prometheus, OpenNebula thông qua các công cụ chuẩn hóa.

Trong hệ thống, CrewAI điều phối các Agent sử dụng MCP Server để thu thập dữ liệu giám sát. Dữ liệu sau đó được Groq API xử lý và suy luận để đánh giá trạng thái tài nguyên, phát hiện bất thường và đưa ra các đề xuất quản lý. Kết quả được trả về Web App để quản trị viên theo dõi và ra quyết định.

Sự kết hợp này giúp tự động hóa quá trình giám sát và phân tích hạ tầng, nâng cao hiệu quả vận hành so với phương pháp thủ công.

2.3.6 Tổng quan về MAPE-K

MAPE-K (Monitor – Analyze – Plan – Execute – Knowledge) là mô hình vòng điều khiển tự trị được IBM đề xuất nhằm xây dựng các hệ thống có khả năng tự giám sát và tự quản lý. Mô hình gồm năm thành phần chính:

- **Monitor:** Thu thập dữ liệu và trạng thái từ hệ thống.
- **Analyze:** Phân tích dữ liệu để phát hiện bất thường hoặc đánh giá hiệu năng.
- **Plan:** Xây dựng kế hoạch hoặc đề xuất hành động phù hợp.
- **Execute:** Thực hiện các hành động đã được lập kế hoạch.
- **Knowledge:** Kho tri thức lưu trữ thông tin về hệ thống, chính sách, ngưỡng tài nguyên và lịch sử hoạt động.

Trong đề tài, MAPE-K được sử dụng làm khung kiến trúc cho hệ thống AI Agent. Dữ liệu giám sát được thu thập từ Prometheus, phân tích bởi AI Agent, từ đó đưa ra đề xuất tối ưu tài nguyên và hỗ trợ thực thi trên hạ tầng Cloud, với toàn bộ quá trình dựa trên kho tri thức dùng chung.

2.4 Web Application

2.4.1 Tổng quan

Web Application là giao diện tương tác giữa quản trị viên và hệ thống giám sát Cloud. Ứng dụng cho phép theo dõi trạng thái tài nguyên, cảnh báo hệ thống và kết quả phân tích từ AI Agent theo thời gian thực. Hệ thống được xây dựng theo mô hình client-server gồm Frontend và Backend.

2.4.2 Frontend

HTML5 và JavaScript: Frontend được xây dựng bằng HTML5 và JavaScript thuần (Vanilla JS), giúp giao diện nhẹ, dễ triển khai và tích hợp với các thành phần giám sát.

Tailwind CSS: Framework CSS theo hướng utility-first, hỗ trợ xây dựng giao diện nhanh chóng và nhất quán.

Google Fonts và Material Symbols: Được sử dụng để cung cấp font chữ và biểu tượng trực quan, nâng cao trải nghiệm người dùng.

2.4.3 Backend

FastAPI: Framework Python hiệu năng cao dùng để xây dựng REST API, hỗ trợ xử lý bất đồng bộ và tự động sinh tài liệu API.

Pydantic: Thư viện dùng để định nghĩa và kiểm tra dữ liệu đầu vào/đầu ra, đảm bảo tính chính xác và nhất quán của các API.

2.5 GitOps trong quản lý cấu hình hệ thống

2.5.1 Tổng quan về GitOps

GitOps là phương pháp quản lý hạ tầng và ứng dụng sử dụng Git làm nguồn lưu trữ trạng thái mong muốn của hệ thống. Mọi thay đổi cấu hình đều được thực hiện thông qua quy trình Git như commit, review và merge, giúp đảm bảo tính minh bạch, khả năng kiểm soát phiên bản và phục hồi hệ thống khi cần thiết.

2.5.2 Ứng dụng GitOps trong đề tài

Trong đề tài, GitOps được sử dụng để quản lý tập trung mã nguồn và cấu hình của hệ thống, bao gồm Backend, Frontend, cấu hình Prometheus, Grafana, AI Agent và các script triển

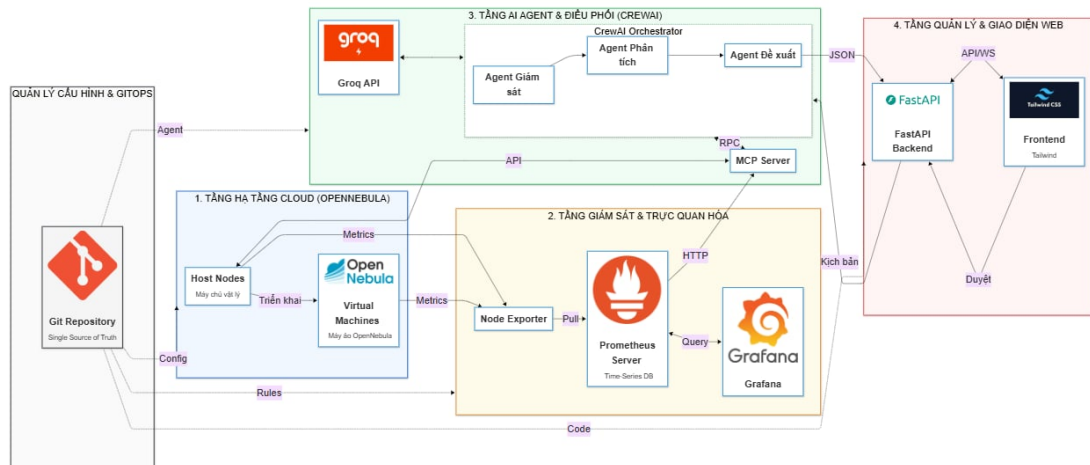
CHƯƠNG 2. CƠ SỞ LÝ THUYẾT

khai. Nhờ đó, toàn bộ hệ thống có thể được theo dõi, cập nhật và tái triển khai một cách nhất quán từ repository Git.

Chương 3. PHƯƠNG PHÁP THỰC HIỆN

Chương này trình bày chi tiết các phương pháp được áp dụng trong quá trình thiết kế, triển khai và đánh giá hệ thống quản lý tài nguyên Cloud thông minh. Mỗi phương pháp được trình bày rõ ràng về lý do lựa chọn, cách thức áp dụng cụ thể và mối liên hệ với các thành phần khác trong hệ thống. Các phương pháp được tổ chức thành sáu nhóm chính: phương pháp triển khai hệ thống, phương pháp giám sát tài nguyên, phương pháp tích hợp AI theo vòng điều khiển MAPE-K, phương pháp thiết kế và xây dựng Web App, phương pháp quản lý cấu hình bằng GitOps và phương pháp kiểm tra đánh giá chất lượng hệ thống.

3.1 Mô hình triển khai hệ thống



Hình 3.1. Mô hình kiến trúc tổng thể của hệ thống

3.2 Phương pháp triển khai hệ thống

3.2.1 Kiến trúc triển khai tổng thể

Hệ thống được xây dựng theo kiến trúc bốn lớp chức năng:

- **Lớp hạ tầng (Infrastructure Layer):** Bao gồm môi trường OpenNebula, các Host Node và Node Exporter dùng để thu thập dữ liệu tài nguyên như CPU, RAM, Disk và Network.
- **Lớp giám sát (Monitoring Layer):** Prometheus thu thập và lưu trữ dữ liệu giám sát, Alertmanager xử lý cảnh báo, trong khi Grafana trực quan hóa các chỉ số hệ thống.

- **Lớp xử lý AI (AI Processing Layer):** MCP Server cung cấp các công cụ truy cập Prometheus và OpenNebula. CrewAI phối hợp các AI Agent sử dụng Groq API để phân tích tài nguyên theo mô hình MAPE-K và đưa ra đề xuất tối ưu.
- **Lớp giao diện (Presentation Layer):** FastAPI cung cấp các API và xử lý nghiệp vụ, trong khi giao diện Web hiển thị dữ liệu giám sát và kết quả phân tích cho quản trị viên. ““

3.2.2 Quy trình triển khai

Hệ thống được triển khai theo các bước sau:

- **Bước 1:** Cài đặt và cấu hình môi trường OpenNebula, Host Node và hệ thống quản lý mã nguồn Git.
- **Bước 2:** Triển khai Node Exporter, Prometheus và Grafana để thu thập và trực quan hóa dữ liệu giám sát.
- **Bước 3:** Xây dựng MCP Server và các công cụ truy cập Prometheus API, OpenNebula API.
- **Bước 4:** Tích hợp CrewAI và Groq API, xây dựng các Agent thực hiện giám sát, phân tích và đề xuất tối ưu tài nguyên.
- **Bước 5:** Phát triển Web Application, tích hợp hệ thống giám sát và AI Agent, sau đó kiểm thử toàn bộ hệ thống.

3.3 Phương pháp giám sát hệ thống

3.3.1 Thu thập dữ liệu

Hệ thống sử dụng Prometheus theo mô hình pull-based để thu thập dữ liệu từ Node Exporter trên các Host Node. Dữ liệu được lưu trữ dưới dạng chuỗi thời gian và cung cấp cho Grafana cũng như AI Agent thực hiện phân tích.

Các thông số giám sát chính gồm:

- **Scrape Interval:** 15 giây.
- **Retention Time:** 15 ngày.

- **Ngưỡng cảnh báo:**

- ◇ CPU sử dụng > 85
- ◇ RAM khả dụng < 10
- ◇ Dung lượng đĩa sử dụng > 90

3.3.2 Các chỉ số giám sát về ngưỡng cảnh báo

Bảng 3.1. Các chỉ số giám sát và ngưỡng cảnh báo của hệ thống

Chỉ số	Nguồn	Ngưỡng WARNING	Ngưỡng CRITICAL
CPU Usage (%)	Node Exporter	> 85% trong 5 phút	> 95% trong 2 phút
RAM Available	Node Exporter	< 15% tổng RAM	< 5% tổng RAM
Disk Usage (%)	Node Exporter	> 80% dung lượng	> 90% dung lượng
Network I/O	Node Exporter	> 80% băng thông	> 95% băng thông
CPU Load Average	Node Exporter	> số lõi $\times$ 1.5	> số lõi $\times$ 2.0

3.3.3 Phương pháp trực quan hóa với Grafana

Grafana được sử dụng làm lớp hiển thị chính với ba nhóm dashboard:

- **Dashboard tổng quan (System Overview):** Trạng thái toàn bộ cluster, tỷ lệ sử dụng tài nguyên theo từng Host Node và danh sách cảnh báo đang hoạt động.
- **Dashboard chi tiết theo node:** Biểu đồ chuỗi thời gian chi tiết cho CPU, RAM, Disk I/O và Network của từng Host Node, phục vụ phân tích sâu khi xảy ra sự cố.
- **Dashboard AI Agent:** Hiển thị kết quả phân tích, xu hướng dự đoán và trạng thái thực thi các hành động điều chỉnh của AI Agent.

Các Grafana panel được nhúng trực tiếp vào Web App thông qua iframe embed, giúp quản trị viên không cần truy cập riêng Grafana mà vẫn đủ thông tin trong một giao diện thống nhất.

3.4 Phương pháp tích hợp AI theo kiến trúc MAPE-K

3.4.1 Ánh xạ MAPE-K vào các thành phần hệ thống

Bảng 3.2. Mô hình hoạt động MAPE-K trong hệ thống

Giai đoạn	Thành phần	Hoạt động cụ thể
Monitor (M)	Prometheus + Node Exporter	Thu thập liên tục chỉ số CPU, RAM, Disk, Network từ Host Node mỗi 15 giây. Alertmanager kích hoạt AI Agent khi phát hiện vi phạm ngưỡng cảnh báo.
Analyze (A)	Monitor Agent (CrewAI + Gemini)	Agent truy vấn dữ liệu từ Prometheus qua MCP Server. Gemini API phân tích xu hướng, phát hiện bất thường và đánh giá mức độ nghiêm trọng của tình trạng tài nguyên hiện tại.
Plan (P)	Analyst Agent (CrewAI + Gemini)	Dựa trên kết quả phân tích, Agent đề xuất biện pháp điều chỉnh cụ thể, ưu tiên hóa theo mức độ tác động và rủi ro. Kế hoạch được trình bày dưới dạng JSON có cấu trúc kèm mô tả ngôn ngữ tự nhiên.
Execute (E)	Reporter Agent + FastAPI	Sau khi quản trị viên phê duyệt (hoặc tự động trong chế độ autonomous), hệ thống thực thi kế hoạch qua API tương ứng. Kết quả được ghi log vào Knowledge Base.
Knowledge (K)	TSDB Prometheus + Log hệ thống	Lưu trữ ngưỡng tài nguyên, quy tắc phân tích, lịch sử hành động và kết quả điều chỉnh. AI Agent tham chiếu liên tục để cải thiện độ chính xác theo thời gian.

3.4.2 Luồng hoạt động chi tiết

Vòng lặp MAPE-K trong hệ thống được thực hiện theo trình tự:

- **Thu thập (Monitor):** Hệ thống kích hoạt AI Agent theo lịch định kỳ hoặc khi Alertmanager phát cảnh báo. Monitor Agent gọi tool MCP Server để lấy snapshot dữ liệu tài nguyên hiện tại từ Prometheus.
- **Phân tích (Analyze):** Gemini API tiếp nhận dữ liệu tài nguyên cùng Knowledge base (ngưỡng, lịch sử), suy luận để xác định trạng thái bình thường hay bất thường, nguồn gốc vấn đề và xu hướng trong 15–30 phút tới.
- **Lập kế hoạch (Plan):** Analyst Agent đề xuất biện pháp điều chỉnh cụ thể, ưu tiên

hóa theo mức tác động và rủi ro. Kế hoạch được trình bày dạng JSON có cấu trúc kèm giải thích ngôn ngữ tự nhiên.

- **Chờ phê duyệt (Pending Approval):** Kế hoạch gửi lên Web App, quản trị viên xem xét và nhấn Approve hoặc Reject. Ở chế độ autonomous, bước này bỏ qua nếu hành động nằm trong phạm vi an toàn định sẵn.
- **Thực thi (Execute):** Reporter Agent thực thi kế hoạch đã phê duyệt, gọi API tương ứng và theo dõi kết quả. Toàn bộ hành động được ghi log chi tiết.
- **Cập nhật kiến thức (Knowledge Update):** Kết quả thực thi được phân tích và lưu vào Knowledge base, làm dữ liệu tham chiếu cho các vòng lặp tiếp theo, giúp AI Agent cải thiện dần theo thời gian.

3.4.3 Chế độ hoạt động của AI Agent

Hệ thống hỗ trợ hai chế độ linh hoạt, chuyển đổi qua endpoint POST `/api/agent/toggle-autonomous`:

- **Chế độ Manual (Human-in-the-loop):** Kế hoạch khắc phục của AI sẽ bị tạm dừng ở trạng thái chờ duyệt (WAITING_APPROVAL). Nó đóng vai trò là Hệ thống hỗ trợ ra quyết định (DSS). Quản trị viên (Human Admin) phải đọc log và nhập lệnh vào Chatbox thì AI mới được phép tiến hành sửa lỗi.
- **Chế độ Autonomous (Tự động):** AI Agent tự động thực thi trong phạm vi hành động an toàn được định nghĩa trước (safe action boundary). Các hành động rủi ro cao vẫn yêu cầu phê duyệt thủ công.

3.5 Phương pháp thiết kế và xây dựng Web App

Web App được thiết kế theo phương pháp component-based, phân tách rõ ràng giữa Frontend và Backend. Nguyên tắc thiết kế chủ đạo là đơn giản, trực quan và cập nhật thời gian thực, đảm bảo quản trị viên có thể nắm bắt trạng thái hệ thống nhanh chóng mà không cần thao tác phức tạp.

3.5.1 Kiến trúc tổng thể Web App

Web App được tổ chức theo mô hình Client–Server với hai thành phần độc lập:

- **Frontend (Client):** Xây dựng bằng HTML5, JavaScript thuần (Vanilla JS) và Tailwind CSS. Giao tiếp với Backend qua Fetch API bất đồng bộ (async/await). Tự động cập nhật dữ liệu mỗi 5 giây bằng vòng lặp setInterval(), không cần tải lại trang.
- **Backend (Server):** Xây dựng bằng FastAPI (Python), tổ chức theo router-based với các nhóm endpoint phân tách theo chức năng. Sử dụng Pydantic để validate dữ liệu đầu vào, hỗ trợ xử lý bất đồng bộ với async/await.

3.5.2 Phương pháp thiết kế giao diện Frontend

Nguyên tắc thiết kế:

- **Real-time first:** Mọi dữ liệu giám sát đều được cập nhật tự động. Fetch API được gọi định kỳ 5 giây để đồng bộ trạng thái mà không ảnh hưởng trải nghiệm người dùng.
- **Information hierarchy:** Thông tin được phân cấp rõ ràng: 4 chỉ số tổng quan (metrics cards) ở trên cùng, biểu đồ chi tiết ở giữa, danh sách cảnh báo và trạng thái node ở cột phải.
- **Dark mode:** Giao diện tối mặc định (class dark trên thẻ html) phù hợp với môi trường vận hành hệ thống, giảm mỏi mắt khi theo dõi liên tục.
- **Responsive layout:** Sử dụng hệ thống grid của Tailwind CSS với các breakpoint sm, md, lg để giao diện thích nghi với nhiều kích thước màn hình.

Bảng 3.3. Cấu trúc giao diện hệ thống

Trang	Nội dung và chức năng
Dashboard	Hiển thị 4 chỉ số tổng quan (total instances, system health, network I/O, security incidents), danh sách Critical Alerts theo thời gian thực, trạng thái Host Node và 3 Grafana panel nhúng (CPU Load Distribution, RAM Utilization, Network Latency).
AI Agent	Theo dõi vòng đời AI Agent theo cấu trúc MAPE-K: hiển thị phase hiện tại (Monitor/Analyze/Plan/Execute), kết quả phân tích và đề xuất điều chỉnh, nút Approve/Reject kế hoạch, toggle chế độ autonomous và nút trigger kiểm thử.
System Logs	Xem lịch sử vận hành hệ thống: log thu thập dữ liệu, kết quả phân tích AI Agent, hành động điều chỉnh đã thực hiện và kết quả tương ứng.

3.5.3 Phương pháp xây dựng Backend API

Backend được xây dựng theo kiến trúc RESTful API với FastAPI, tổ chức module hóa theo từng nhóm chức năng:

- **Router phân tách:** Dashboard router (/api/dashboard/) cung cấp dữ liệu tổng quan. Agent router (/api/agent/) quản lý vòng đời AI Agent. Mỗi router là một module Python độc lập, dễ bảo trì và mở rộng.
- **Async/await:** Tất cả endpoint sử dụng async def để xử lý bất đồng bộ, đảm bảo Backend không bị block khi AI Agent đang xử lý hoặc khi gọi Prometheus API.
- **Pydantic validation:** Các request model (như ActionReq) được khai báo kế thừa BaseModel, FastAPI tự động validate dữ liệu đầu vào và trả về lỗi 422 có cấu trúc khi dữ liệu không hợp lệ.
- **State management:** Trạng thái AI Agent được lưu trong object agent_instance (singleton pattern), chia sẻ giữa các endpoint qua cơ chế dependency injection của FastAPI.

3.6 Phương pháp quản lý cấu hình bằng GitOps

GitOps được áp dụng làm phương pháp quản lý cấu hình và triển khai hệ thống, sử dụng Git làm nguồn sự thật duy nhất (single source of truth). Mọi thay đổi về cấu hình hạ tầng, mã nguồn ứng dụng hay tham số vận hành đều thực hiện qua quy trình Git (commit → pull request → review → merge), không can thiệp trực tiếp vào môi trường đang chạy.

Git repository được tổ chức theo cấu trúc thư mục rõ ràng, phân tách theo từng thành phần hệ thống:

- **backend/:** Mã nguồn FastAPI, các router, model Pydantic, logic khởi tạo AI Agent và kết nối Prometheus API.
- **frontend/:** File HTML, JavaScript và cấu hình Tailwind CSS của giao diện Web App.
- **monitoring/:** File prometheus.yml, cấu hình scrape targets, alerting rules và Grafana dashboard JSON.
- **ai_agent/:** Định nghĩa Agent, Task, Crew trong CrewAI và cấu hình MCP Server tools.

CHƯƠNG 3. PHƯƠNG PHÁP THỰC HIỆN

- **scripts/**: Script cài đặt môi trường, khởi tạo dịch vụ và cấu hình hạ tầng OpenNebula.

3.7 Phương pháp kiểm tra và đánh giá hệ thống

Bảng 3.4. Các kịch bản kiểm thử hệ thống

STT	Kịch bản kiểm thử	Kết quả kỳ vọng	Tiêu chí đạt
1	Tạo tình huống tải cao CPU (> 90%) trên Host Node	Prometheus cảnh báo, Alertmanager kích hoạt, AI Agent phân tích và đề xuất điều chỉnh	Hoàn thành < 2 phút
2	Mô phỏng RAM thiếu hụt bằng cách tạo nhiều máy ảo	AI Agent phát hiện xu hướng RAM giảm, đề xuất tái phân bổ hoặc tắt máy ảo không cần thiết	Đề xuất đúng và khả thi
3	Quản trị viên phê duyệt kế hoạch điều chỉnh trên Web App	FastAPI nhận lệnh approve, Agent chuyển sang Execute, kết quả hiển thị trên giao diện	Thực thi thành công
4	Bật chế độ Autonomous và kích hoạt anomaly	AI Agent tự phân tích, lập kế hoạch và thực thi mà không cần phê duyệt thủ công	Hoàn chỉnh < 3 phút
5	Ngắt kết nối Prometheus giữa chừng	Web App hiển thị lỗi, Agent dừng phân tích, hệ thống tự phục hồi khi Prometheus hoạt động lại	Xử lý lỗi đúng
6	Thay đổi cấu hình Agent qua Git và triển khai lại	Cấu hình mới được áp dụng đúng, Agent hoạt động theo tham số mới, không ảnh hưởng dữ liệu cũ	GitOps rollout thành công

Bảng 3.5. Các chỉ tiêu đánh giá hệ thống

CHƯƠNG 3. PHƯƠNG PHÁP THỰC HIỆN

Chỉ tiêu	Mục tiêu	Phương pháp đo
Độ trễ phát hiện bất thường	< 2 phút	Đo từ lúc sự kiện xảy ra đến khi kết quả hiển thị trên Web App
Độ chính xác đề xuất AI	> 80% đề xuất phù hợp	Quản trị viên đánh giá từng đề xuất trên tập 20 kịch bản kiểm thử
Thời gian phản hồi API	< 500ms (P95)	Đo bằng công cụ kiểm thử HTTP với 100 request đồng thời
Tính liên tục giám sát	> 99% uptime	Tỷ lệ scrape thành công / tổng số lần scrape theo lịch
Tài nguyên tự tiêu thụ	< 10% tài nguyên cluster	Đo CPU/RAM của Prometheus, Grafana, AI Agent trong quá trình vận hành

Chương 4. TRIỂN KHAI HỆ THỐNG

Chương này trình bày chi tiết các bước thao tác kỹ thuật để thiết lập môi trường, cấu hình mạng, cài đặt các dịch vụ lõi và chạy thử nghiệm hệ thống quản trị đám mây tự trị. Quá trình triển khai được tiến hành tuần tự từ lớp hạ tầng ảo hóa thấp nhất (OpenNebula, KVM), lớp giám sát đo lường (Prometheus), cho đến lớp ứng dụng Trí tuệ nhân tạo (AI Agent, FastMCP). Cuối chương là phần kiểm thử hệ thống dựa trên các kịch bản thực tế nhằm đánh giá hiệu năng và tính đúng đắn của toàn bộ mô hình thiết kế.

4.1 Môi trường và mô hình triển khai

Để mô phỏng kiến trúc của một trung tâm dữ liệu quy mô nhỏ, hệ thống được triển khai trên môi trường ảo hóa với mô hình Cụm tối thiểu bao gồm 02 máy chủ. Cả hai máy chủ đều vận hành trên nền tảng hệ điều hành Ubuntu 20.04 LTS để đảm bảo tính ổn định và tương thích thư viện tốt nhất.

Bảng 4.1. Quy hoạch tài nguyên và địa chỉ IP hệ thống

Vai trò	Hostname	IP Address	Cấu hình tài nguyên	Nhiệm vụ chính
Front-end Node	controller-node	192.168.57.7	4 vCPU, 8GB RAM, 50GB Disk	Cài đặt OpenNebula (oned, sunstone), Prometheus Server, Backend FastAPI, AI Worker, MCP Server
Host Node	compute-node-01	192.168.57.9	4 vCPU, 8GB RAM, 100GB Disk	Chạy lớp ảo hóa KVM, triển khai và vận hành các máy ảo

4.2 Triển khai nền tảng Cloud OpenNebula

4.2.1 Thiết lập kho lưu trữ và cập nhật hệ thống

Trước khi cài đặt bất kỳ gói phần mềm nào, cả hai máy chủ cần được cập nhật chỉ mục và cấu hình để nhận diện kho lưu trữ chính thức của OpenNebula phiên bản 6.x. Tại Terminal của cả hai máy, thực thi các lệnh sau:

CHƯƠNG 4. TRIỂN KHAI HỆ THỐNG

```
# Cập nhật danh sách các gói phần mềm hệ điều hành
sudo apt update && sudo apt upgrade -y

# Tải và thêm khóa GPG để xác thực các gói OpenNebula
wget -q -O- https://downloads.opennebula.io/repo/repo.key |
sudo apt-key add -

# Thêm kho lưu trữ OpenNebula cho Ubuntu 20.04
echo "deb https://downloads.opennebula.io/repo/6.4/Ubuntu/20.04
stable main" \
| sudo tee /etc/apt/sources.list.d/opennebula.list

# Cập nhật lại chỉ mục apt
sudo apt update
```

4.2.2 Triển khai nút quản trị (Front-end Node)

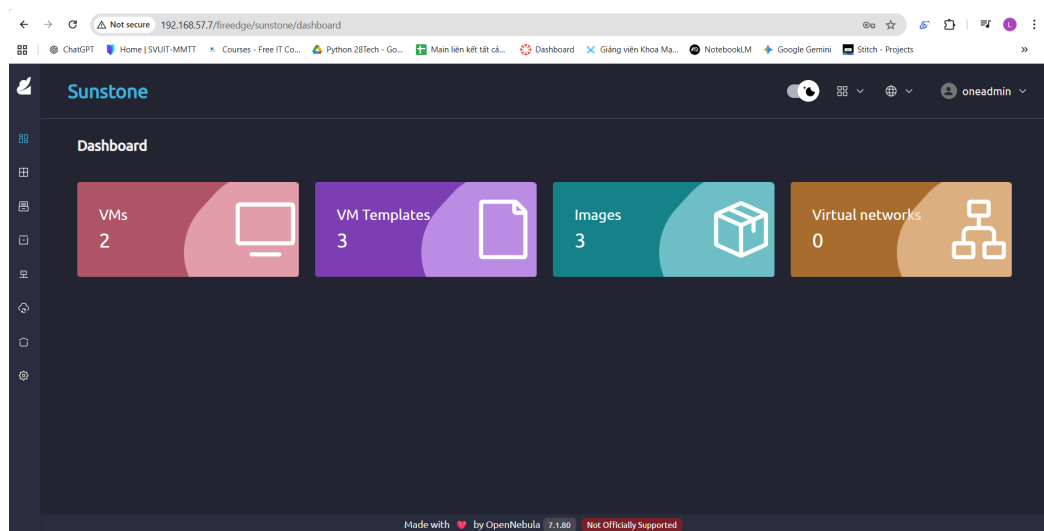
Máy chủ controller-node (192.168.57.7) đóng vai trò là "bộ não" của hạ tầng ảo hóa. Tiến hành cài đặt các dịch vụ daemon cốt lõi bao gồm trình quản lý oned, giao diện web sunstone và các dịch vụ mạng nâng cao:

```
# Cài đặt các thành phần Front-end
sudo apt install opennebula opennebula-sunstone opennebula-gate
opennebula-flow -y

# Cài đặt các thư viện Ruby cần thiết cho giao diện Sunstone
sudo /usr/share/one/install_gems

# Khởi động và cấu hình dịch vụ tự động chạy khi khởi động lại máy
sudo systemctl enable opennebula opennebula-sunstone
sudo systemctl start opennebula opennebula-sunstone
```

Truy cập vào địa chỉ: <http://192.168.57.7/fireedge/sunstone/dashboard> để xem giao diện sau khi đã cài đặt thành công.



Hình 4.1. Giao diện của OpenNebula Frontend

4.2.3 Triển khai nút tính toán KVM (Host Node) và cấu hình xác thực

Trên máy chủ compute-node-01 (192.168.57.9), tiến hành cài đặt môi trường ảo hóa KVM để biến máy tính này thành một nút cung cấp tài nguyên:

```
# Cài đặt gói OpenNebula Node và hypervisor KVM
```

```
sudo apt install opennebula-node-kvm -y
```

```
# Kích hoạt dịch vụ libvirtd quản lý máy ảo
```

```
sudo systemctl enable libvirtd
```

```
sudo systemctl start libvirtd
```

Để Front-end có thể ra lệnh cho Host Node mà không bị chặn bởi tường lửa hoặc yêu cầu nhập mật khẩu liên tục, hệ thống bắt buộc phải thiết lập cơ chế Xác thực SSH bằng khóa đối với tài khoản oneadmin. Từ Terminal của Front-end Node, thực hiện:

```
# Chuyển quyền sang người dùng quản trị hệ thống OpenNebula
```

```
su - oneadmin
```

```
# Khởi tạo cặp khóa Public/Private RSA
```

```
ssh-keygen -t rsa
```

```
# Đẩy khóa Public sang Host Node để cấp quyền truy cập
```

```
ssh-copy-id oneadmin@192.168.57.9
```

CHƯƠNG 4. TRIỂN KHAI HỆ THỐNG

```
# Cấu hình file known_hosts để bỏ qua cảnh báo xác thực lần đầu
ssh-keyscan 192.168.57.9 >> ~/.ssh/known_hosts
```

4.3 Triển khai kiến trúc giám sát

Hệ thống AI không thể hoạt động nếu thiếu dữ liệu đầu vào. Do đó, lớp giám sát Prometheus được thiết lập để liên tục đo lường mức độ tiêu thụ tài nguyên của các máy chủ.

4.3.1 Cài đặt tác nhân thu thập dữ liệu (Node Exporter)

Tại cả hai máy chủ (Front-end và Host Node), cài đặt prometheus-node-exporter. Đây là một dịch vụ chạy ngầm làm nhiệm vụ đọc các thông số cấp thấp từ hệ điều hành và phơi bày qua giao thức HTTP:

```
sudo apt install prometheus-node-exporter -y
sudo systemctl enable prometheus-node-exporter
sudo systemctl start prometheus-node-exporter
```

Quá trình này sẽ mở cổng 9100 trên mỗi máy. Dữ liệu thô lúc này đã sẵn sàng để được thu thập.

4.3.2 Cấu hình máy chủ Prometheus trung tâm

Tại Front-end Node, tiến hành cài đặt máy chủ Prometheus để định kỳ kéo dữ liệu từ các Exporter vừa thiết lập:

```
sudo apt install prometheus -y
```

Tiến hành chỉnh sửa tệp cấu hình lõi của Prometheus bằng lệnh:

sudo nano /etc/prometheus/prometheus.yml. Cấu hình chu kỳ kéo dữ liệu (scrape_interval) là 15 giây và khai báo danh sách IP của toàn bộ hệ thống:

```
global:
  scrape_interval: 15s

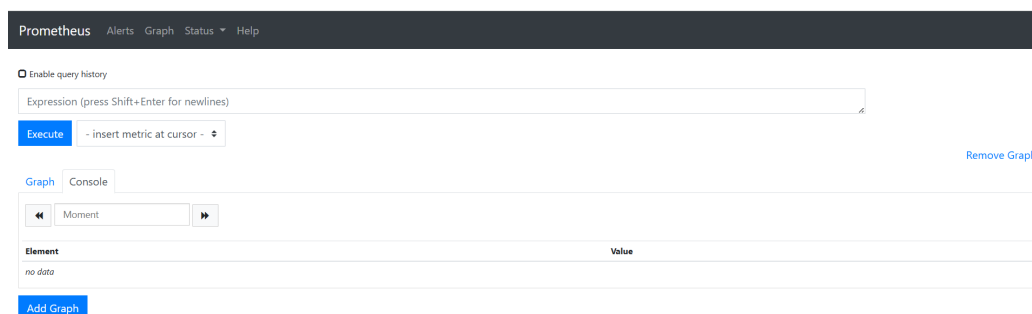
scrape_configs:
  - job_name: 'openebula_cluster'
    static_configs:
      - targets: ['localhost:9100', '192.168.57.9:9100']
```

CHƯƠNG 4. TRIỂN KHAI HỆ THỐNG

Sau khi lưu tệp, khởi động lại dịch vụ để Prometheus áp dụng quy tắc mới:

```
sudo systemctl restart prometheus
```

Truy cập: `[http://192.168.57.7:9090](http://192.168.57.7:9090)` để viết các câu lệnh PromQL kiểm tra xem dữ liệu CPU, RAM đã được thu thập.



Hình 4.2. Giao diện của Prometheus

Endpoint	State	Labels	Last Scrape	Scrape Duration	Error
http://10.0.100.226:8000/metrics	UP	<code>instance="10.0.100.226:8000"</code> <code>job="aiops-backend"</code> <code>role="ai-agent"</code> <code>service="nebulastack-aiops"</code>	4.669s ago	6.963ms	
alertmanager (1/1 up) show less					
Endpoint	State	Labels	Last Scrape	Scrape Duration	Error
http://localhost:9093/metrics	UP	<code>instance="localhost:9093"</code> <code>job="alertmanager"</code> <code>service="alertmanager"</code>	1.962s ago	12.52ms	
node (1/1 up) show less					
Endpoint	State	Labels	Last Scrape	Scrape Duration	Error
http://192.168.57.9:9100/metrics	UP	<code>instance="192.168.57.9:9100"</code> <code>job="node"</code> <code>nodename="host-node"</code> <code>role="opennebula-host"</code>	9.121s ago	324.1ms	
prometheus (1/1 up) show less					
Endpoint	State	Labels	Last Scrape	Scrape Duration	Error
http://localhost:9090/metrics	UP	<code>instance="localhost:9090"</code> <code>job="prometheus"</code> <code>service="prometheus"</code>	1.153s ago	9.336ms	

Hình 4.3. Xây dựng hệ thống thu thập dữ liệu từ Prometheus

4.3.3 Triển khai Grafana Dashboard

Trong khi Prometheus đóng vai trò thu thập và lưu trữ, Grafana được tích hợp để cung cấp khả năng trực quan hóa dữ liệu chuyên nghiệp, giúp quản trị viên theo dõi biến động tài nguyên qua các biểu đồ đồ họa. Tại Front-end Node, thực hiện cài đặt phiên bản ổn định của Grafana:

CHƯƠNG 4. TRIỂN KHAI HỆ THỐNG

```
# Thêm khóa GPG và kho lưu trữ Grafana
```

```
wget -q -O- https://packages.grafana.com/gpg.key | sudo apt-key add -
```

```
sudo add-apt-repository "deb https://packages.grafana.com/oss/deb stable"
sudo apt update && sudo apt install grafana -y
```

```
# Khởi động dịch vụ
```

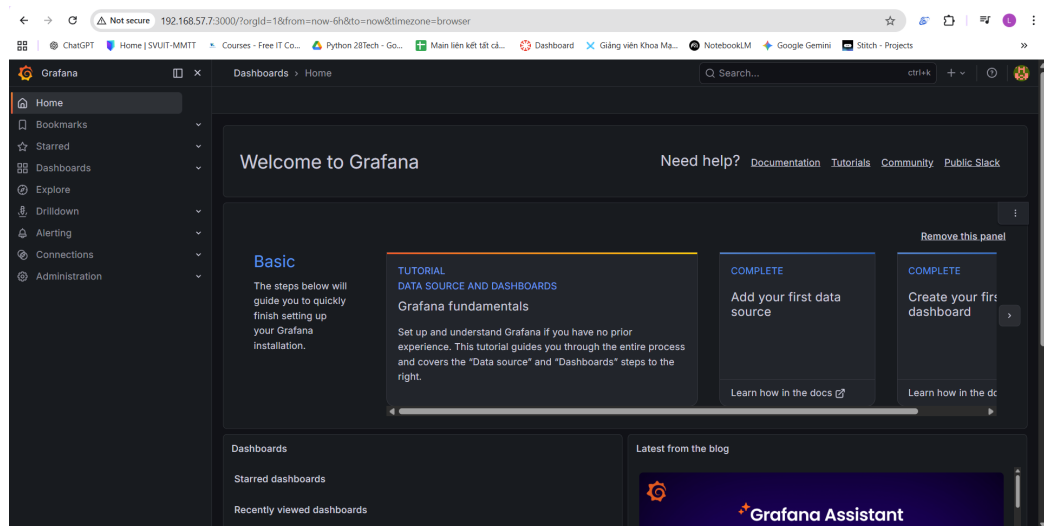
```
sudo systemctl enable grafana-server
```

```
sudo systemctl start grafana-server
```

Kết nối: Truy cập <http://192.168.57.7:3000> (tài khoản mặc định admin/admin). Tiến hành thêm Data Source là Prometheus với địa chỉ <http://localhost:9090>.

Trực quan hóa: Nhập (Import) Dashboard mẫu mã hiệu 1860 (Node Exporter Full). Dashboard này cung cấp cái nhìn toàn diện về CPU Load, RAM Usage và Network Traffic của cả Front-end và Host Node theo thời gian thực.

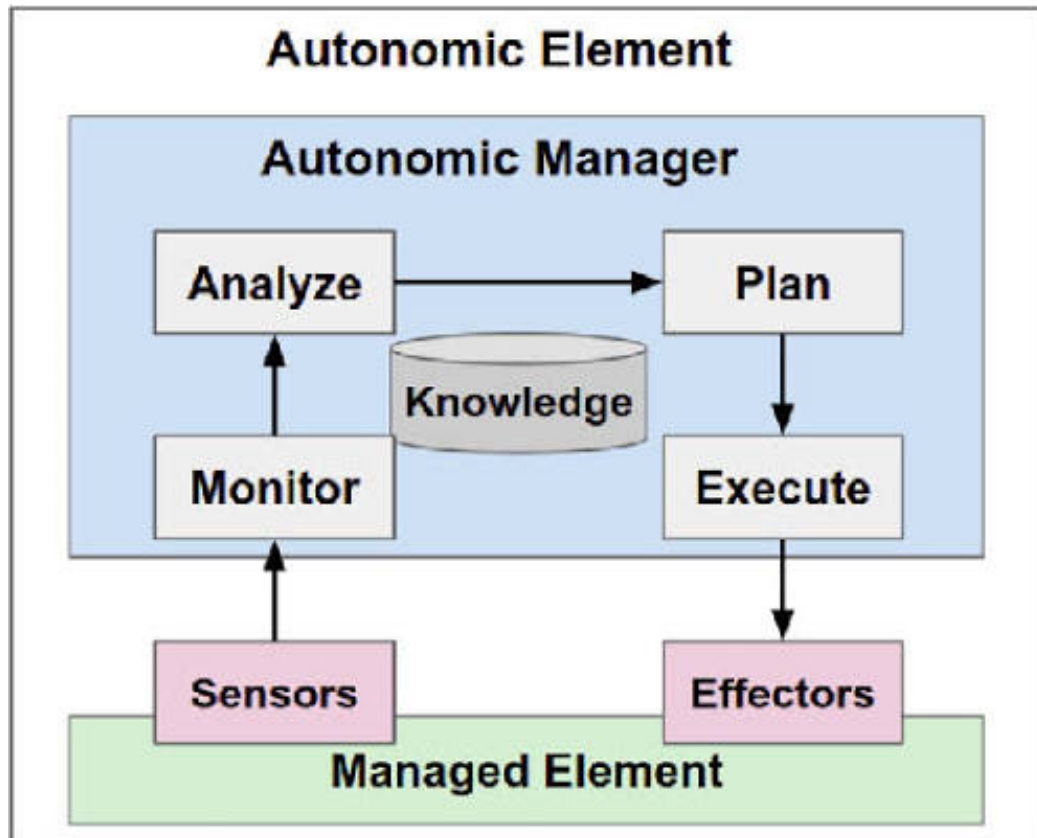
Ý nghĩa: Đây là công cụ đối chứng quan trọng. Khi AI Agent báo cáo có sự cố, quản trị viên có thể kiểm tra chéo các biểu đồ trên Grafana để xác nhận tính chính xác của dữ liệu.



Hình 4.4. Giao diện của Grafana

4.4 Triển khai AI Agent theo kiến trúc MAPE-K

Kiến trúc MAPE-K (Monitor - Analyze - Plan - Execute - Knowledge) được áp dụng để xây dựng luồng xử lý tự động cho AI Agent. Mỗi giai đoạn trong chu kỳ được thiết kế để đảm bảo hệ thống có khả năng tự điều chỉnh và tối ưu hóa tài nguyên Cloud.



Hình 4.5. Kiến trúc MAPE-K

4.4.1 Giai đoạn Monitor & Analyze: Thu thập và Đánh giá chỉ số

Đoạn code sau thể hiện cách chuyển đổi dữ liệu thô từ Prometheus thành các cảnh báo (Alerts) có ý nghĩa.

```
def _evaluate(self, snap: MetricSnapshot) -> list:
    """Đánh giá các chỉ số dựa trên ngưỡng cấu hình (Thresholds)"""
    alerts = []
    thr = THRESHOLDS

    # Kiểm tra ngưỡng CPU (Time-based threshold)
    if snap.cpu_pct >= thr["cpu"]["warning"]:
        # Logic kiểm tra thời gian duy trì trạng thái quá tải
        # ... (phần code xử lý elapsed_time)
        alerts.append({
            "metric": "cpu",
            "level": "CRITICAL",
            "message": f"CPU quá tải ({snap.cpu_pct:.1f}%) liên tục"
```

```
    })

    # Kiểm tra ngưỡng RAM
    if snap.ram_avail_pct <= thr["ram"]["critical"]:
        alerts.append({
            "metric": "ram",
            "level": "CRITICAL",
            "message": f"Dung lượng RAM trống cực thấp:
            {snap.ram_avail_pct:.1f}%"
        })

    return alerts
```

4.4.2 Giai đoạn Plan: Khởi tạo AI Agent

Đoạn code thể hiện việc tích hợp CrewAI và Groq API (Llama 3.3).

```
from crewai import Agent, Task, Crew, LLM

# Khởi tạo mô hình ngôn ngữ lớn (LLM) qua Groq
worker_llm = LLM(
    model="groq/llama-3.3-70b-versatile",
    api_key=os.getenv("GROQ_API_KEY"),
    temperature=0.0
)

# Cấu hình Agent đóng vai trò kiến trúc sư xử lý sự cố
plan_agent = Agent(
    role="Remediation Architect",
    goal="Đưa ra chính xác 3 bước xử lý sự cố ưu tiên theo định dạng
    backstory="Bạn là một chuyên gia SRE cấp cao chuyên vận hành hạ
    llm=worker_llm
)

# Định nghĩa Task với yêu cầu định dạng JSON
```

```
task = Task(
    description=f"Dữ liệu lỗi: {alert_block}. Hãy lập kế hoạch xử lý
    expected_output="Một mảng JSON chứa các bước: step, action, targ
    agent=plan_agent
)
```

4.4.3 Kỹ thuật Adapter: Giải quyết xung đột Schema

```
def adapt_mcp_tool(langchain_tool):
    """
    Bọc Tool của MCP thành BaseTool của CrewAI để xử lý
    xung đột JSON Schema của Groq API.
    """
    class AdaptedTool(BaseTool):
        name: str = langchain_tool.name
        description: str = langchain_tool.description
        args_schema: type[BaseModel] = create_model(...)

        def _run(self, **kwargs):
            return langchain_tool.invoke(kwargs)

    return AdaptedTool()
```

4.4.4 Giai đoạn Execute: Phê duyệt và Thực thi tự động

Giai đoạn Execute chịu trách nhiệm thực thi các hành động được đề xuất bởi AI Agent sau khi hoàn thành bước lập kế hoạch. Để đảm bảo an toàn cho môi trường Cloud, hệ thống được thiết kế theo cơ chế Human-in-the-Loop, trong đó mọi thay đổi tác động đến hạ tầng đều cần được quản trị viên xem xét và phê duyệt thông qua Web Application hoặc AI Chatbox trước khi thực hiện.

Sau khi nhận được tín hiệu phê duyệt, AI Agent sẽ sử dụng MCP Tool để gửi các lệnh tương ứng đến OpenNebula hoặc các thành phần hạ tầng liên quan. Kết quả thực thi của từng hành động được ghi nhận và chuyển tiếp sang giai đoạn Knowledge để lưu trữ.

Đoạn code dưới đây minh họa quá trình thực thi kế hoạch xử lý sự cố:

```
async def execute_remediation_plan(self, plan: list, user_action: st
```



```
"""Thực thi kế hoạch xử lý sự cố sau khi có lệnh phê duyệt"""
```

```
if user_action == "dismiss":
    self.current_phase = "MONITOR"
    return {
        "status": "cancelled",
        "message": "Kế hoạch đã bị hủy."
    }

execution_results = []

# Duyệt qua từng bước trong kế hoạch
for step in plan:
    action = step.get("action")
    target = step.get("target")

    # Gọi MCP Client để tương tác với hạ tầng
    result = await mcp_client.call_tool(
        name="execute_infrastructure_action",
        arguments={
            "action": action,
            "target": target
        }
    )

    execution_results.append({
        "step": step["step"],
        "result": result
    })

self.current_phase = "KNOWLEDGE"
return execution_results
```

4.4.5 Giai đoạn Knowledge: Lưu trữ tri thức và cập nhật ngữ cảnh

Knowledge là thành phần trung tâm của kiến trúc MAPE-K, đóng vai trò lưu trữ toàn bộ thông tin liên quan đến quá trình vận hành hệ thống. Dữ liệu được ghi nhận bao gồm các cảnh báo được phát hiện, kế hoạch xử lý do AI Agent sinh ra, kết quả thực thi và trạng thái cuối cùng của sự cố.

Thông tin này được lưu trữ trong cơ sở dữ liệu SQLite và ChromaDB nhằm phục vụ cho mục đích kiểm toán hệ thống, theo dõi lịch sử hoạt động và cung cấp ngữ cảnh cho các vòng phân tích tiếp theo. Việc duy trì kho tri thức giúp AI Agent đưa ra các quyết định nhất quán và chính xác hơn trong quá trình vận hành.

Đoạn code sau minh họa việc lưu trữ dữ liệu sự cố vào cơ sở dữ liệu:

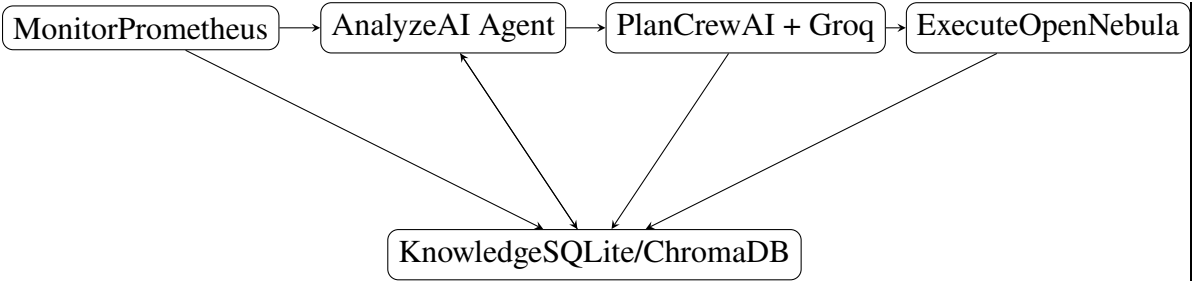
```
async def save_knowledge_base(
    self,
    incident_data: dict,
    results: list
):
    """Lưu trữ lịch sử sự cố vào hệ thống cơ sở dữ liệu"""

    query = """
    INSERT INTO system_logs
    (timestamp, phase, alerts, plan, execute_status)
    VALUES (?, ?, ?, ?, ?)
    """

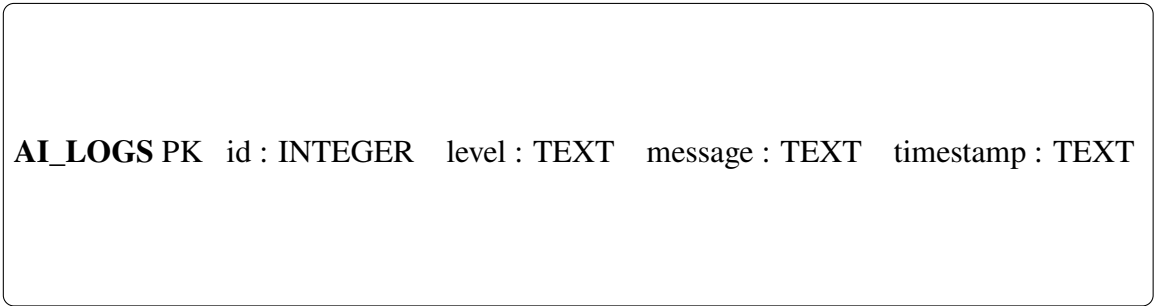
    await db.execute(
        query,
        (
            incident_data['timestamp'],
            'RESOLVED',
            json.dumps(incident_data['alerts']),
            json.dumps(incident_data['plan']),
            json.dumps(results)
        )
    )
```

)

```
await db.commit()
```



Hình 4.6. Chu trình MAPE-K trong hệ thống giám sát và quản lý tài nguyên Cloud



Hình 4.7. Mô hình database bảng AI_LOGS

4.5 Thiết kế và triển khai ứng dụng Web quản trị

Để đáp ứng tiêu chí giám sát trực quan và tương tác thông minh (Human-in-the-Loop), một ứng dụng Web dạng Single Page Application (SPA) đã được phát triển và tích hợp liền mạch vào hệ thống Backend.

Bảng 4.2. Các thư viện chính và chức năng trong hệ thống

Thư viện	Phiên bản	Chức năng
FastAPI	0.135.3	Xây dựng REST API backend bất đồng bộ cho hệ thống
Uvicorn	0.44.0	ASGI server để triển khai ứng dụng FastAPI
CrewAI	1.14.4	Framework đa tác nhân phục vụ xử lý AI
MCP	1.26.0	Cơ chế ủy quyền thực thi giúp AI Agent tương tác an toàn với hạ tầng
Chromadb	1.1.1	Vector database lưu trữ embedding và tri thức
aiosqlite	0.21.0	Lưu trữ dữ liệu hệ thống và lịch sử hoạt động
HTTTPX	0.28.1	Gửi request HTTP bất đồng bộ
Pydantic	2.12.5	Kiểm tra và validate dữ liệu đầu vào
PyJWT	2.12.1	Xác thực và phân quyền người dùng bằng JWT
OpenTelemetry	1.34.1	Thu thập và giám sát telemetry hệ thống

4.5.1 Kiến trúc giao diện và lựa chọn công nghệ Front-end

Giao diện AIOps Dashboard được lập trình dựa trên triết lý "nhẹ và tối ưu tốc độ" (Lightweight & Performant).

- **Ngôn ngữ thiết kế:** Bố cục dạng lưới được chia làm 3 cột chức năng rõ ràng. Giao diện sử dụng nền tảng chế độ tối nhằm giảm thiểu nhức mắt cho quản trị viên khi trực hệ thống ban đêm.
- **TailwindCSS:** Toàn bộ CSS được xử lý thông qua TailwindCSS dạng nhúng, cho phép tạo các thành phần UI/UX trực tiếp trên file HTML mà không làm phình to dung lượng tải trang.
- **Vanilla JavaScript:** Logic xử lý sự kiện và gọi API được viết bằng JS thuần (ES6+),

loại bỏ sự phụ thuộc vào các framework nặng như React hay Vue, giúp giao diện phản hồi gần như tức thời.

4.5.2 Phân hệ giám sát và kỹ thuật đồng bộ dữ liệu

Phân hệ giám sát sử dụng cơ chế Polling để cập nhật dữ liệu theo thời gian thực. Thay vì yêu cầu người dùng tải lại trang, hàm `setInterval` trong JavaScript sẽ định kỳ 2 giây gửi HTTP GET Request tới *endpoint* `/api/logs` của Backend.

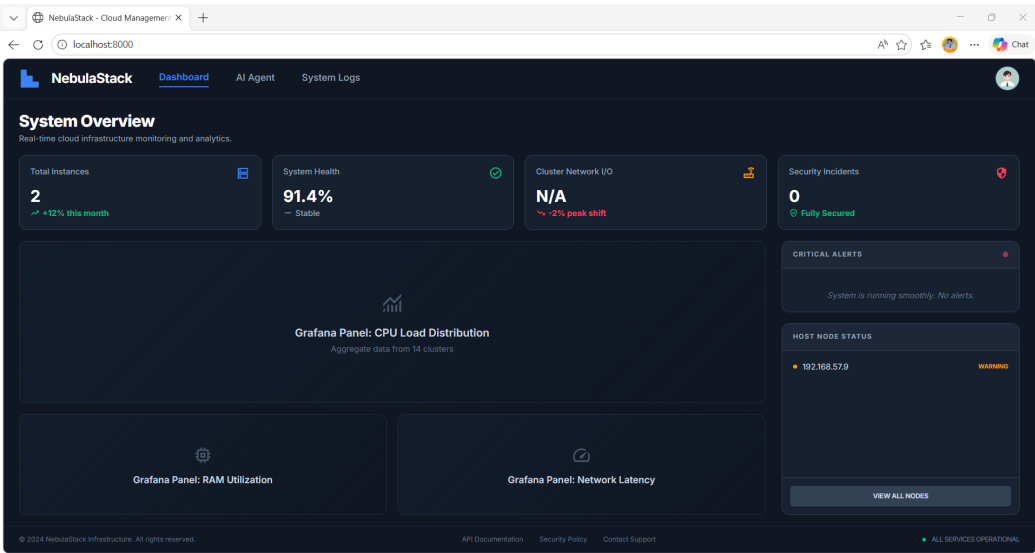
- **Xử lý dữ liệu trả về:** Dữ liệu JSON từ Backend được phân tích và hiển thị theo mức độ nghiêm trọng (INFO, WARNING, ERROR, SUCCESS). Hệ thống sử dụng các lớp màu Tailwind tương ứng để giúp quản trị viên dễ dàng nhận biết trạng thái của từng sự kiện.
- **Tự động cuộn trang (Auto-scroll):** Khi xuất hiện log mới, thuộc tính `element.scrollTop` = `element.scrollHeight` được kích hoạt để tự động cuộn xuống cuối khung hiển thị, đảm bảo người dùng luôn theo dõi được các sự kiện mới nhất.

4.5.3 Xử lý Logic Chatbox và hiển thị đề xuất AI

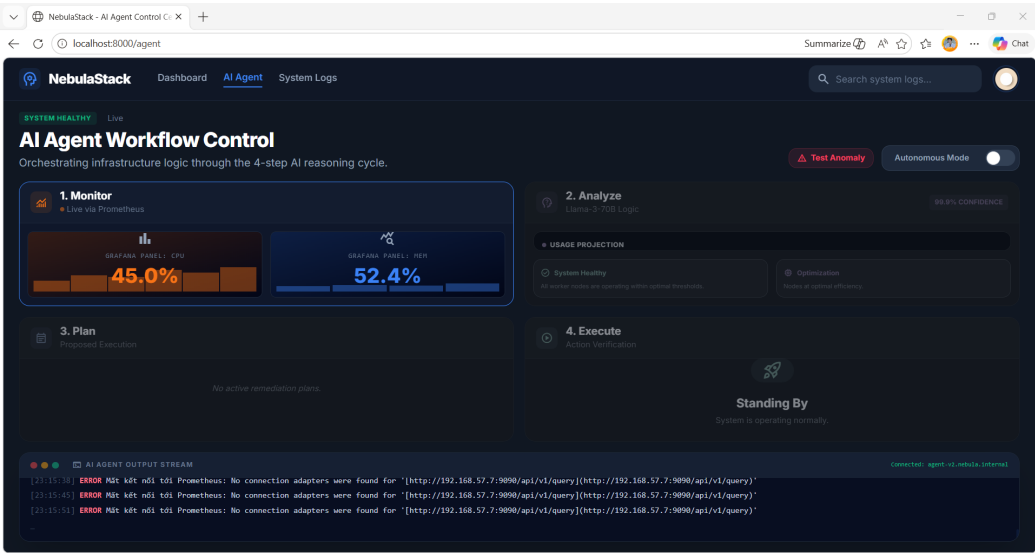
Khung Chatbox là thành phần trung tâm của giai đoạn EXECUTE trong kiến trúc MAPE-K, cho phép quản trị viên tương tác trực tiếp với AI Agent.

- **Gửi lệnh tương tác:** Khi người dùng nhập lệnh (ví dụ: “Phê duyệt kế hoạch, scale node 01”), JavaScript sẽ đóng gói nội dung thành payload và gửi HTTP POST tới *endpoint* `/api/chat`.
- **Kiểm soát trạng thái giao diện:** Trong thời gian chờ phản hồi, hệ thống hiển thị hiệu ứng Loading Spinner và tạm thời vô hiệu hóa nút gửi nhằm tránh các yêu cầu trùng lặp.
- **Hiển thị kết quả AI:** Các phản hồi từ LLM như kế hoạch xử lý sự cố, bảng Markdown hoặc dữ liệu JSON được định dạng lại để hiển thị rõ ràng, giúp quản trị viên dễ dàng đánh giá và đưa ra quyết định phê duyệt.

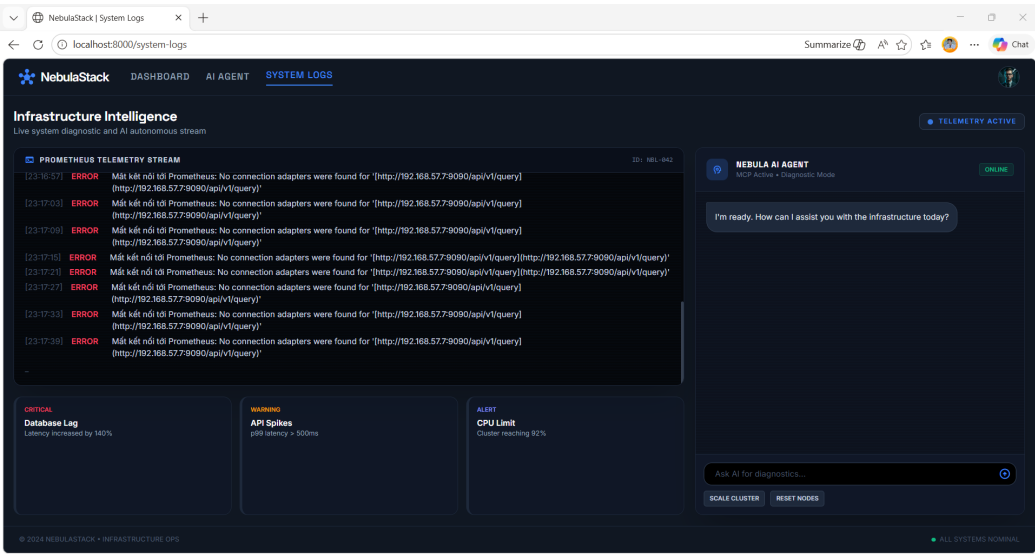
CHƯƠNG 4. TRIỂN KHAI HỆ THỐNG



Hình 4.8. Giao diện Web của trang Dashboard



Hình 4.9. Giao diện Web của trang AI Agent



Hình 4.10. Giao diện Web của trang System Log & AI Chatbox

4.6 Tích hợp CrewAI và giao thức MCP

Để hệ thống AI Agent có thể suy luận logic và tương tác trực tiếp với nền tảng ảo hóa OpenNebula một cách an toàn, đề án kết hợp hai công nghệ cốt lõi: CrewAI đóng vai trò "bộ não" điều phối các tác nhân và MCP đóng vai trò "cánh tay" giao tiếp với hạ tầng vật lý.

4.6.1 Điều phối kịch bản tự động hóa với CrewAI

CrewAI được sử dụng để định nghĩa một đội ngũ chuyên gia ảo vận hành theo cơ chế nhập vai. Thay vì sử dụng một mô hình ngôn ngữ lớn nguyên khối để xử lý mọi thứ—điều rất dễ dẫn đến hiện tượng ảo giác và xuất ra các lệnh sai lệch—hệ thống chia nhỏ công việc thành các tác vụ chuyên biệt.

Việc gom nhóm các Agent và Task được quản lý bởi đối tượng Crew, cho phép thiết lập quy trình làm việc tuần tự. Kết quả đầu ra của Agent phân tích sẽ làm đầu vào cho Agent lập kế hoạch xử lý sự cố.

```
from crewai import Crew, Process

# Khởi tạo đội ngũ Agent xử lý sự cố hạ tầng
remediation_crew = Crew(
    agents=[monitor_agent, plan_agent],
    tasks=[analyze_task, create_plan_task],
    process=Process.sequential,
    verbose=True
)

# Kích hoạt quá trình làm việc của AI
result = remediation_crew.kickoff()
```

4.6.2 Đóng gói công cụ hạ tầng qua FastMCP Server

Một rủi ro bảo mật rất lớn khi tích hợp AI vào quản trị hệ thống là việc cấp quyền thực thi lệnh trực tiếp (arbitrary code execution). Để giải quyết vấn đề này, đề án ứng dụng chuẩn giao thức mã nguồn mở MCP thông qua thư viện FastMCP.

CHƯƠNG 4. TRIỂN KHAI HỆ THỐNG

MCP cung cấp một lớp bảo mật không gian Sandboxing. Thay vì cho phép AI tự do gõ lệnh bash vào máy chủ, hệ thống định nghĩa trước các công cụ có giới hạn. AI chỉ được phép gọi các hàm này thông qua cơ chế Function Calling.

Đoạn code sau minh họa việc khởi tạo MCP Server và phơi bày các công cụ quản trị OpenNebula an toàn:

```
from fastmcp import FastMCP

# Khởi tạo MCP Server quản lý hạ tầng
mcp = FastMCP("OpenNebula_AIOps_Server")

@mcp.tool()
def restart_virtual_machine(vm_id: int) -> str:
    """Khởi động lại một máy ảo trên OpenNebula khi bị treo."""
    # Gọi API XML-RPC của OpenNebula để restart VM
    # ...
    return f"Đã gửi lệnh khởi động lại máy ảo ID {vm_id}"

@mcp.tool()
def get_node_status(node_id: int) -> dict:
    """Lấy thông tin trạng thái hiện tại của một Host Node."""
    # Truy xuất database hoặc OpenNebula API
    # ...
    return {"node_id": node_id, "status": "ACTIVE", "cpu_usage": "95"}
```

4.6.3 Cơ chế giao tiếp khép kín giữa LLM và Hệ thống

Sự kết hợp giữa CrewAI và MCP tạo ra một vòng lặp giao tiếp chặt chẽ:

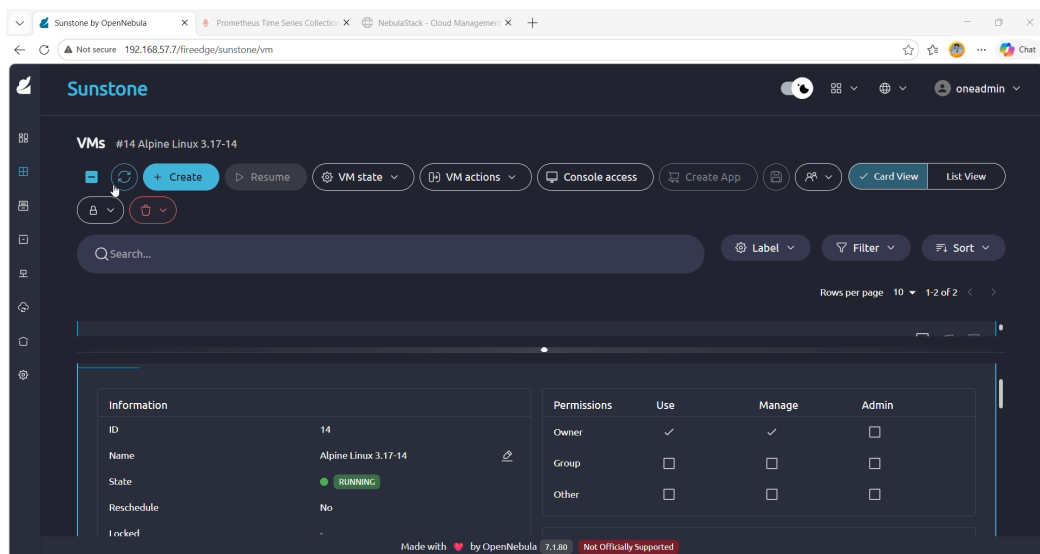
1. **Phát yêu cầu:** Khi `plan_agent` của CrewAI nhận thấy cần thu thập thêm thông tin hoặc cần thay đổi trạng thái máy chủ, nó sẽ sinh ra một yêu cầu dưới định dạng JSON (Function Call).
2. **Thực thi an toàn:** Yêu cầu này được chuyển đến FastMCP Server đang chạy trên controller-node. Máy chủ MCP sẽ kiểm tra tính hợp lệ của tham số, ánh xạ tới hàm Python tương ứng (ví dụ: `restart_virtual_machine`) và thực thi hàm.

CHƯƠNG 4. TRIỂN KHAI HỆ THỐNG

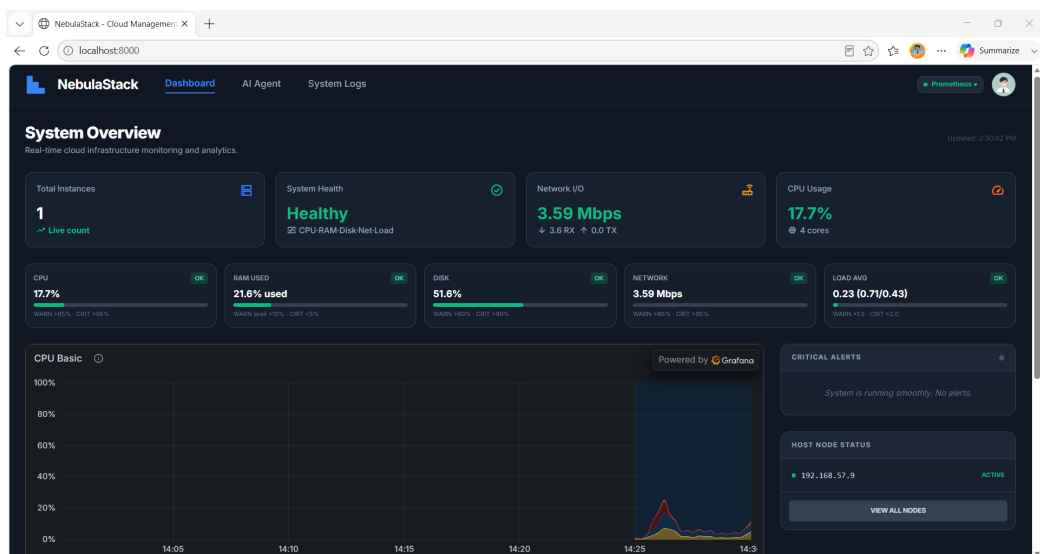
3. **Phản hồi ngữ cảnh:** Kết quả thực thi (hoặc thông báo lỗi nếu có) được đóng gói lại và trả về cho mô hình LLM. Dựa vào kết quả này, AI có thể quyết định bước hành động tiếp theo hoặc báo cáo hoàn thành nhiệm vụ.

Kiến trúc này đảm bảo tính "Plug-and-play", cho phép dễ dàng mở rộng thêm các công cụ quản trị mới mà không cần phải thay đổi logic cốt lõi của AI Agent.

4.7 Kiểm thử và đánh giá kết quả hệ thống



Hình 4.11. Kiểm tra chức năng tạo các VM

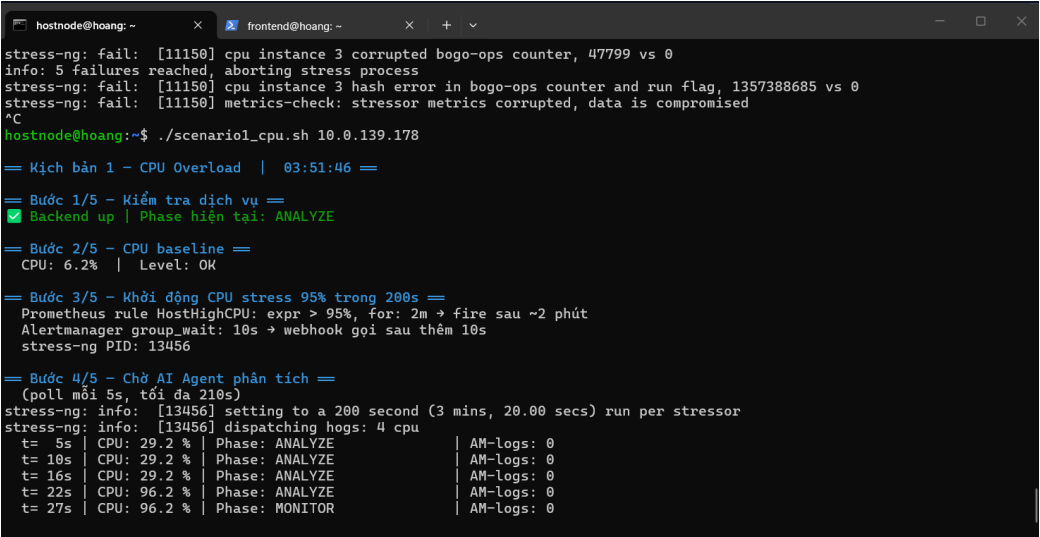


Hình 4.12. Các thông số của hệ thống

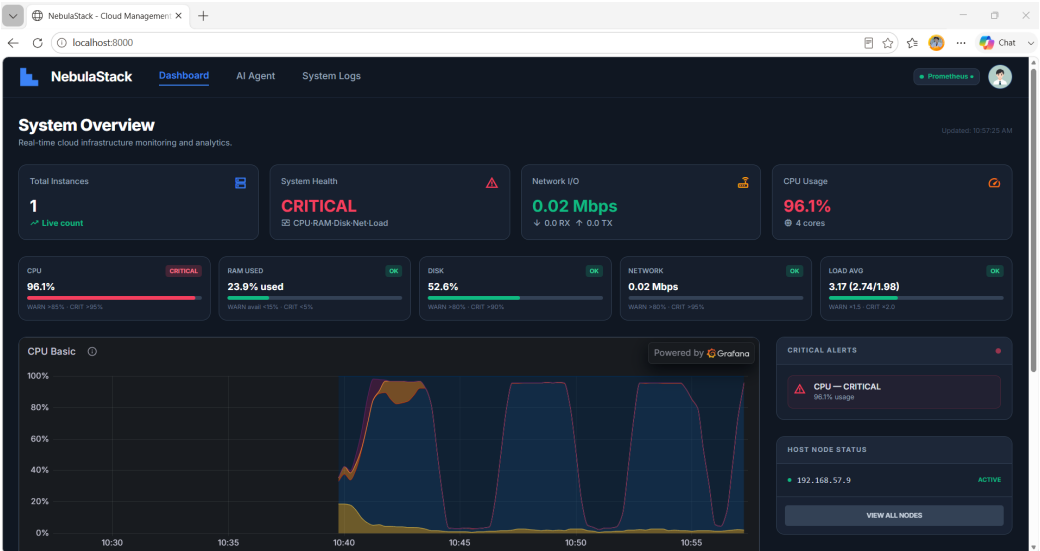
4.7.1 Kịch bản kiểm thử

1. **Mục đích:** Đánh giá độ nhạy của hệ thống giám sát và tốc độ lập kế hoạch xử lý (vòng lặp Monitor - Analyze - Plan) của AI Agent khi Host Node bị cạn kiệt tài nguyên. Yêu cầu hệ thống phải phản hồi dưới 2 phút.
2. **Công cụ thực hiện:** Sử dụng công cụ `stress-ng` trên Host Node để ép mức sử dụng CPU lên 95%.
3. **Tiến trình thực thi:**
 - **Bước 1 - Khởi tạo:** Xác nhận AI Agent đang ở trạng thái rảnh rỗi và ghi nhận mức CPU cơ sở.
 - **Bước 2 - Tạo tải giả lập:** Chạy lệnh `stress-ng` ép CPU hoạt động ở mức 95% liên tục trong 200 giây nhằm kích hoạt quy tắc cảnh báo `HostHighCPU` của Prometheus.
 - **Bước 3 - Theo dõi tín hiệu:** Kịch bản liên tục truy vấn (poll) hệ thống mỗi 5 giây để theo dõi biến động biểu đồ và chờ Webhook từ Alertmanager.
 - **Bước 4 - Xuất bản kế hoạch:** Ngay khi nhận cảnh báo, AI Agent tiến hành phân tích và trả về bản kế hoạch xử lý định dạng JSON, hệ thống tự động chuyển sang trạng thái chờ phê duyệt (`WAITING_APPROVAL`).
4. **Tiêu chí thành công:** Alertmanager đẩy Webhook chính xác; AI Agent nhận diện đúng sự cố và đưa ra giải pháp khắc phục khả thi (ví dụ: Di dời máy ảo, mở rộng tài nguyên) trong khoảng thời gian quy định (< 2 phút).

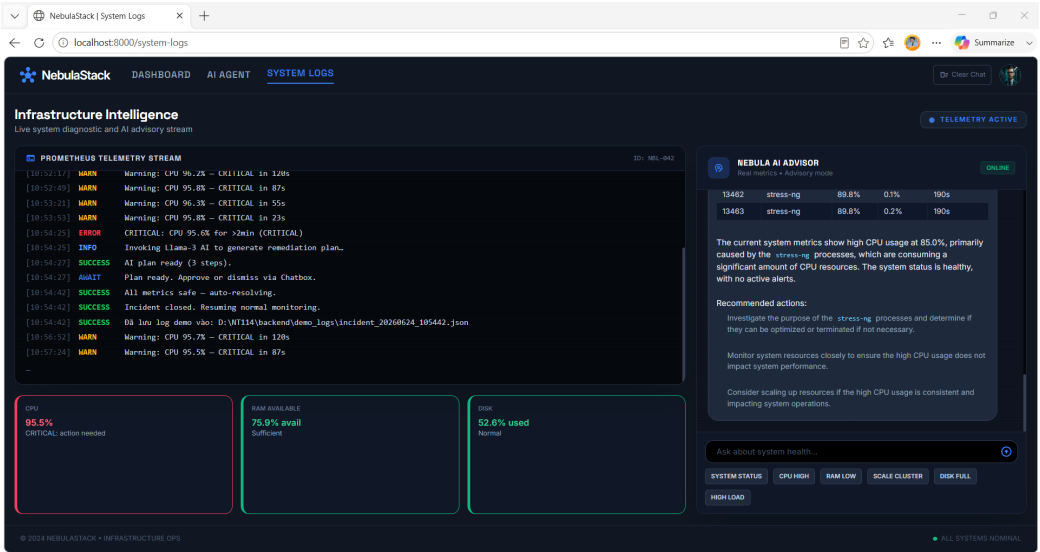
CHƯƠNG 4. TRIỂN KHAI HỆ THỐNG



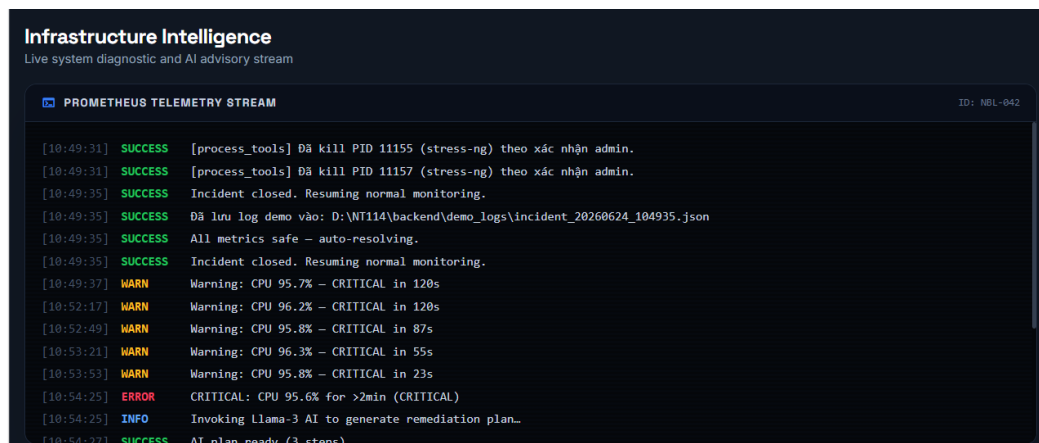
Hình 4.13. Chạy kịch bản stress-ng trên Host Node



Hình 4.14. Dashboard hệ thống báo critical khi chạy kịch bản



Hình 4.15. Chat với AI để tìm process và yêu cầu kill process



Hình 4.16. Kết quả sau khi kill process

4.7.2 Đánh giá kết quả

Đánh giá kết quả chi tiết: Kịch bản kiểm thử số 1 đã chứng minh tính hiệu quả và sự ưu việt của việc ứng dụng chu trình MAPE-K kết hợp cùng AI Agent vào công tác SRE. Kết quả thực thi được ghi nhận chi tiết qua các khía cạnh sau:

- **Khả năng chẩn đoán nguyên nhân gốc rễ:** Thay vì phụ thuộc vào các kịch bản IF-ELSE được lập trình cứng nhắc, ngay khi tiếp nhận luồng Webhook từ Alertmanager, AI Agent đã tận dụng khả năng suy luận của mô hình ngôn ngữ lớn để phân tích ngữ cảnh từ log hệ thống. AI đã tự động khoanh vùng và chỉ đích danh tiến trình `stress-ng` đang là "thủ phạm" gây thất cố chai CPU.
- **Lập kế hoạch chuẩn xác:** AI Agent xuất bản thành công bản kế hoạch khắc phục sự cố dưới định dạng JSON có cấu trúc chặt chẽ. Đáng chú ý, hệ thống không chỉ đưa ra lời khuyên chung chung mà xác định chính xác hành động cần thiết: tìm kiếm PID của tiến trình dị thường và thực hiện can thiệp ở cấp độ hệ điều hành.
- **Thực thi dứt điểm và an toàn qua MCP:** Thông qua cơ chế phê duyệt, hệ thống đã gọi thành công bộ công cụ của MCP Server để tự động thực thi lệnh ngắt (`kill`) tiến trình lỗi. Quá trình này diễn ra hoàn toàn cô lập, tiêu diệt chính xác luồng xử lý gây quá tải mà không làm gián đoạn hay ảnh hưởng đến các máy ảo khác đang vận hành chung trên Host Node.
- **Tốc độ phục hồi:** Ngay sau khi công cụ MCP báo cáo thực thi thành công, giám sát trên Grafana cho thấy chỉ số CPU lập tức giảm thẳng đứng từ ngưỡng nguy hiểm 95% về lại mức cơ sở an toàn. Toàn bộ vòng đời sự cố—from khâu phát hiện, phân tích, ra quyết định đến dọn dẹp tài nguyên—được hệ thống tự trị hoàn tất gọn gàng trong thời gian dưới 2 phút.

Chương 5. KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

Chương này tổng kết các kết quả đạt được trong quá trình thực hiện đồ án, đối chiếu với các mục tiêu đã đề ra, đồng thời trình bày những hạn chế còn tồn tại và đề xuất các hướng phát triển trong tương lai.

5.1 Kết quả đạt được

Sau quá trình nghiên cứu, thiết kế và triển khai, đồ án đã hoàn thành các mục tiêu chính đặt ra ban đầu.

Về hạ tầng Cloud: Hệ thống đã triển khai thành công một cụm OpenNebula tối thiểu gồm Front-end Node và Host Node KVM, hỗ trợ quản lý và phân bổ tài nguyên máy ảo theo mô hình IaaS. Cơ chế xác thực SSH giữa các node được cấu hình hoàn chỉnh, cho phép Front-end điều khiển Host Node một cách tự động và an toàn.

Về hệ thống giám sát: Hệ thống giám sát được xây dựng dựa trên Prometheus và Node Exporter, thu thập các chỉ số CPU, RAM, Disk, Network I/O và Load Average theo chu kỳ 15 giây. Các ngưỡng cảnh báo WARNING và CRITICAL được định nghĩa rõ ràng cho từng chỉ số. Alertmanager được tích hợp để gửi cảnh báo tới Backend thông qua webhook, giúp hệ thống phản ứng nhanh khi xảy ra sự cố. Grafana được nhúng trực tiếp vào Web App nhằm hỗ trợ quản trị viên theo dõi và đối chứng dữ liệu trực quan.

Về AI Agent theo kiến trúc MAPE-K: Đồ án đã hiện thực hóa đầy đủ năm thành phần của mô hình MAPE-K gồm Monitor, Analyze, Plan, Execute và Knowledge. CrewAI được sử dụng để điều phối Agent, kết hợp với Groq API và mô hình Llama-3.3-70B nhằm sinh kế hoạch xử lý sự cố dưới dạng JSON có cấu trúc. Hệ thống hỗ trợ hai chế độ vận hành là Manual (Human-in-the-Loop) và Autonomous, cho phép cân bằng giữa tính an toàn và mức độ tự động hóa.

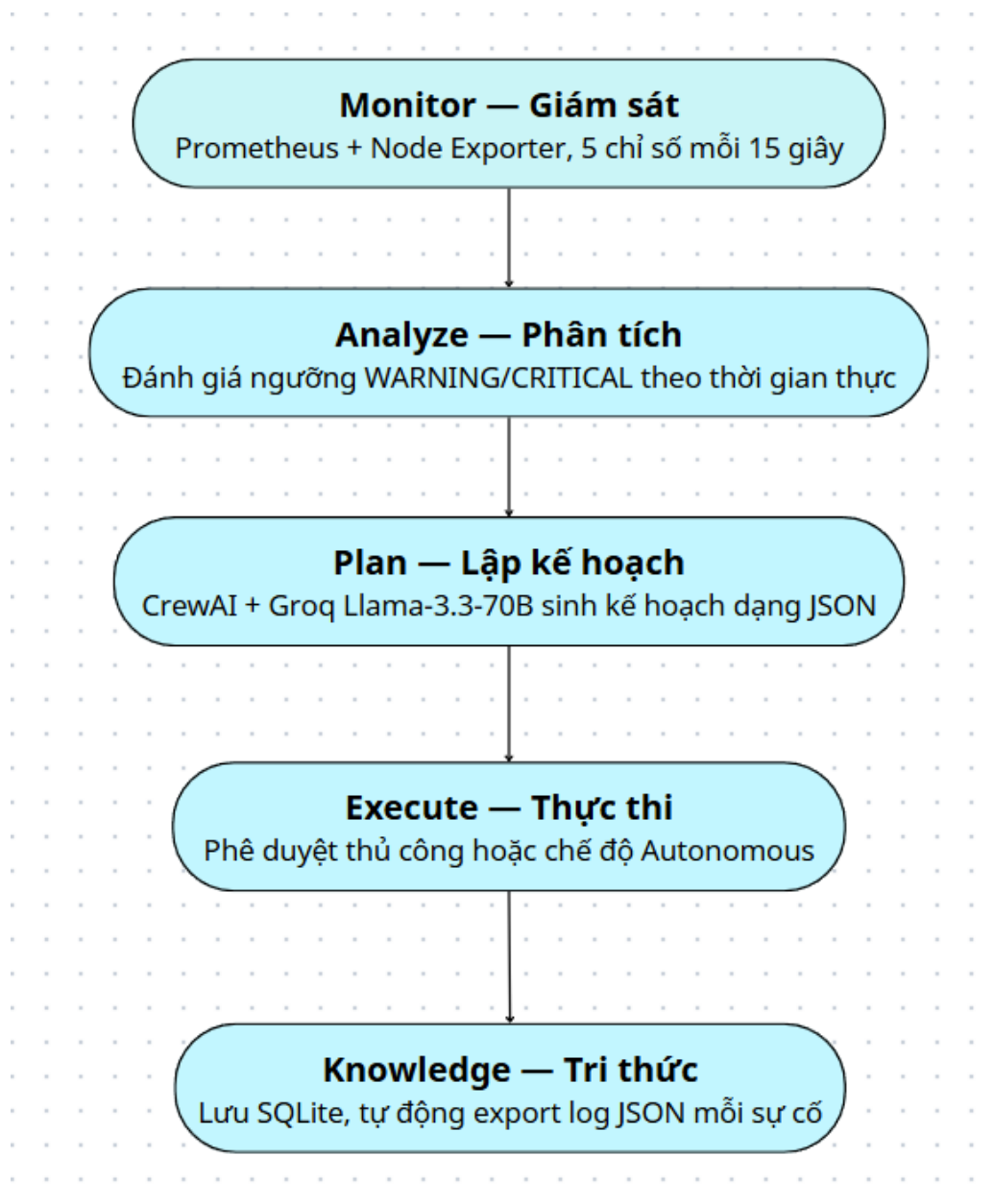
Về khả năng chẩn đoán nâng cao: Hệ thống hỗ trợ truy vấn trực tiếp các tiến trình đang hoạt động trên Host Node thông qua SSH, giúp AI Agent xác định nguyên nhân gây quá tải tài nguyên và đề xuất phương án xử lý phù hợp. Các hành động tác động đến hệ thống đều được bảo vệ bởi cơ chế xác nhận từ quản trị viên nhằm hạn chế rủi ro ngoài ý muốn.

Về ứng dụng Web quản trị: Một Web Application dạng Single Page Application được phát triển bằng FastAPI và JavaScript thuần, bao gồm Dashboard giám sát, AI Agent Dashboard và System Logs kết hợp AI Chatbox. Giao diện được cập nhật theo thời gian

thực thông qua cơ chế polling.

Về quản lý cấu hình: Phương pháp GitOps được áp dụng để quản lý mã nguồn và cấu hình hệ thống, giúp kiểm soát phiên bản, theo dõi thay đổi và đảm bảo tính nhất quán trong quá trình triển khai.

Nhìn chung, hệ thống đã chứng minh được tính khả thi của việc kết hợp Cloud Computing, hệ thống giám sát hiện đại và AI Agent để xây dựng một giải pháp quản lý tài nguyên thông minh, có khả năng giám sát, phân tích và hỗ trợ ra quyết định bán tự động.



Hình 5.1. Tổng kết kết quả đạt được theo mô hình MAPE-K

5.2 Hạn chế của đề tài

Bên cạnh những kết quả đạt được, đề tài vẫn còn tồn tại một số hạn chế:

- Quy mô triển khai còn nhỏ, mới được kiểm thử trên cụm hai node trong môi trường phòng thí nghiệm, chưa đánh giá trên hệ thống lớn hoặc môi trường thực tế.
- Mức độ tự động hóa còn hạn chế, phần lớn các hành động khắc phục vẫn ở mức đề xuất và yêu cầu sự phê duyệt của quản trị viên trước khi thực thi.
- Hệ thống phụ thuộc vào Groq API nên chịu ảnh hưởng bởi giới hạn tốc độ truy cập, độ trễ mạng và tính sẵn sàng của dịch vụ bên thứ ba.
- Độ chính xác của các đề xuất AI mới được kiểm thử trên một số kịch bản giả lập, chưa được đánh giá trên tập dữ liệu sự cố đa dạng trong thời gian dài.
- Chưa tích hợp đầy đủ các cơ chế bảo mật nâng cao như phân quyền người dùng theo vai trò (RBAC), xác thực tập trung hoặc mã hóa toàn bộ luồng giao tiếp nội bộ.

5.3 Hướng phát triển

Trong tương lai, hệ thống có thể được mở rộng theo các hướng sau:

- Mở rộng quy mô triển khai trên nhiều Host Node hơn nhằm đánh giá khả năng mở rộng và hiệu năng trong môi trường thực tế.
- Tăng cường mức độ tự động hóa bằng cách tích hợp thêm các MCP Tool hỗ trợ thực thi trực tiếp các tác vụ như live migration, VM provisioning hoặc cluster scaling trong phạm vi an toàn được kiểm soát.
- Bổ sung các mô hình học máy để dự đoán xu hướng sử dụng tài nguyên, giúp hệ thống chủ động điều chỉnh trước khi xảy ra quá tải.
- Nghiên cứu triển khai các mô hình ngôn ngữ nhỏ (Small LLM) chạy nội bộ nhằm giảm phụ thuộc vào dịch vụ bên ngoài, đồng thời cải thiện độ trễ và tính bảo mật.
- Hoàn thiện cơ sở tri thức bằng cách tích hợp ChromaDB hoặc Vector Database để lưu trữ lịch sử sự cố dưới dạng embedding, hỗ trợ Retrieval-Augmented Generation (RAG) cho AI Agent.

CHƯƠNG 5. KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

- Tăng cường bảo mật thông qua các cơ chế JWT, OAuth2, RBAC và mã hóa các kênh giao tiếp giữa các thành phần trong hệ thống.
- Thực hiện các thử nghiệm dài hạn để đánh giá định lượng các chỉ số như độ trễ phát hiện sự cố, độ chính xác của đề xuất AI và mức tài nguyên tiêu thụ của hệ thống.

Tóm lại, đồ án đã xây dựng thành công một nền tảng quản lý tài nguyên Cloud tích hợp AI Agent, đáp ứng các mục tiêu nghiên cứu đã đề ra. Kết quả đạt được là cơ sở để tiếp tục phát triển theo hướng tự động hóa cao hơn, góp phần thúc đẩy xu hướng ứng dụng AI trong vận hành hạ tầng Cloud và AIOps trong tương lai.

Phụ lục A. CÔNG BỐ KHOA HỌC

Trong phần này, chúng tôi xin đính kèm một bài báo khoa học liên quan đến đề tài khóa luận của chúng tôi:

...

TÀI LIỆU THAM KHẢO

- [1] OpenNebula Systems. *OpenNebula Documentation*. <https://docs.opennebula.io>. 2026.
- [2] Prometheus Authors. *Prometheus Documentation*. <https://prometheus.io/docs>. 2026.
- [3] Grafana Labs. *Grafana Documentation*. <https://grafana.com/docs>. 2026.
- [4] Sebastián Ramírez. *FastAPI Documentation*. <https://fastapi.tiangolo.com>. 2026.
- [5] CrewAI Inc. *CrewAI Documentation*. <https://docs.crewai.com>. 2026.
- [6] Anthropic. *Model Context Protocol Specification*. <https://modelcontextprotocol.io>. 2025.
- [7] Jeffrey O. Kephart và David M. Chess. “The Vision of Autonomic Computing”. In: *Computer* 36.1 (2003), pp. 41–50. DOI: [10.1109/MC.2003.1160055](https://doi.org/10.1109/MC.2003.1160055).
- [8] Michael Wooldridge. *An Introduction to MultiAgent Systems*. 2nd. John Wiley & Sons, 2009.
- [9] Groq Inc. *GroqCloud Documentation*. <https://console.groq.com/docs>. 2026.
- [10] Meta AI. *Llama 3 Models Documentation*. <https://www.llama.com>. 2025.

ĐỀ CƯƠNG CHI TIẾT

TÊN ĐỀ TÀI TIẾNG VIỆT: Thiết kế và triển khai hệ thống quản lý tài nguyên và tự vận hành cho Edge Cloud sử dụng AI Agent đa tác nhân trên cụm máy tính nhúng Raspberry Pi.

TÊN ĐỀ TÀI TIẾNG ANH: Design and Implementation of an Autonomous Resource Management System for Edge Cloud using Multi-Agent AI on a Raspberry Pi Cluster.

Cán bộ hướng dẫn (Học vị, họ và tên, nơi công tác cán bộ hướng dẫn và cán bộ hướng dẫn phụ nếu có): ThS. Trần Thị Dung

Thời gian thực hiện: Từ ngày.....đến ngày.....

Sinh viên thực hiện:

Lê Xuân Hoàng – 23520524

Đặng Minh Dzũ – 23520404

Nội dung đề tài: (Mô tả chi tiết về tổng quan đề tài, mục tiêu, phạm vi, đối tượng, phương pháp thực hiện, kết quả mong đợi của đề tài,...)

Tổng quan đề tài (sinh viên cần nêu rõ các đề tài, sản phẩm liên quan đã được nghiên cứu hoặc đã có trên thị trường trước thời điểm hiện tại và nêu thực trạng của chúng, để từ đó nêu bật được lý do thực hiện nghiên cứu trong KLTN này):

- Trong kỷ nguyên của Internet vạn vật (IoT) và Trí tuệ nhân tạo (AI), xu hướng dịch chuyển từ Điện toán đám mây trung tâm (Centralized Cloud Computing) sang Điện toán biên (Edge Cloud / Edge Computing) đang diễn ra mạnh mẽ nhằm giảm thiểu độ trễ mạng và bảo mật dữ liệu tại nguồn. Tuy nhiên, các hạ tầng Edge Cloud thường được cấu thành từ các thiết bị phần cứng giới hạn năng lượng và tài nguyên (như cụm vi điều khiển, máy tính nhúng Raspberry Pi). Việc quản lý máy ảo, container và điều phối tài nguyên CPU/RAM/Network trên các cụm phần cứng giới hạn này gặp rất nhiều thách thức và phần lớn vẫn phụ thuộc vào cấu hình thủ công của kỹ sư hệ thống (SRE).

Thực trạng các giải pháp liên quan trên thị trường hiện nay:

- Hệ thống lập lịch và quản lý truyền thống (OpenNebula, OpenStack, K3s): Thường sử dụng các thuật toán heuristic hoặc rule-based cố định. Các phương pháp này mang tính phản ứng (reactive), thiếu linh hoạt khi tải hệ thống tại biên biến động đột ngột và chưa tối ưu được cho kiến trúc ARM.
- Hệ thống giám sát và cảnh báo (Prometheus, Grafana, Alertmanager): Thu thập dữ liệu tốt theo thời gian thực nhưng chỉ dừng lại ở mức phát cảnh báo thô, đòi hỏi con người phải can thiệp trực tiếp để cấu hình hoặc sửa lỗi, dễ dẫn đến chậm

trễ gây sập dịch vụ biên.

- Ứng dụng AI/LLM truyền thống trong Quản trị hệ thống (AIOps Chatbot): Các mô hình Chatbot chỉ nhận thông tin và phản hồi bằng văn bản tư vấn, thiếu khả năng tương tác trực tiếp xuống tầng hạ tầng vật lý và thường chạy phụ thuộc vào Cloud API bên thứ ba (như OpenAI, Groq), gây rủi ro lớn về an toàn thông tin và tính độc lập của một mạng Edge nội bộ.

→ Lý do thực hiện nghiên cứu: KLTN này kế thừa nền tảng của đề tài ĐACN (vốn quản lý Cloud bằng AI Agent gọi API Groq trên hạ tầng x86) nhằm giải quyết triệt để các hạn chế trên bằng cách: Dịch chuyển hoàn toàn hạ tầng sang một cụm máy tính nhúng ARM64 (Raspberry Pi Cluster) và nhúng các mô hình ngôn ngữ nhỏ chạy cục bộ (Local Small Language Models - SLM). Hệ thống sẽ hình thành một vòng lặp quản lý tự trị khép kín, an toàn và tối ưu tài nguyên tối đa cho môi trường biên.

Mục tiêu của đề tài (Nêu cụ thể mục tiêu của KLTN, đặc biệt phải nêu được mục tiêu cải tiến sẽ là gì so với thực trạng nêu trong phần tổng quan đề tài):

- Mục tiêu chung: Thiết kế và triển khai một cụm Edge Cloud máy ảo/container tự trị (Autonomous Private Edge Cloud) trên phần cứng Raspberry Pi, được điều phối thông minh bởi hệ thống AI Agent đa tác nhân chạy Local hoàn toàn.

Mục tiêu cải tiến cụ thể so với thực trạng và ĐACN cũ:

- Cải tiến 1 - Chuyển đổi kiến trúc ảo hóa: Chuyển đổi thành công hạ tầng OpenNebula từ kiến trúc x86 truyền thống sang kiến trúc KVM-ARM64 trên cụm Raspberry Pi, tối ưu hóa việc phân cấp tài nguyên siêu nhẹ (MicroVMs/Containers).
- Cải tiến 2 - Nội bộ hóa Trí tuệ nhân tạo (Local AI): Loại bỏ hoàn toàn sự phụ thuộc vào Groq API và mạng Internet bên ngoài. Thay thế bằng một Local SLM (như Llama-3-8B hoặc Qwen-2.5-7B) được lượng tử hóa (Quantized) chạy trực tiếp trong cụm mạng nội bộ, đảm bảo an toàn thông tin tuyệt đối cho Edge Cloud.
- Cải tiến 3 - Nâng cấp vòng lặp tự trị (Fully Autonomous Loop): Mở rộng năng lực xử lý của AI Agent từ mức "đề xuất/kill process cấp OS" lên mức "can thiệp sâu vào hạ tầng Cloud" thông qua giao thức MCP Server cải tiến, thực hiện tự động các tác vụ: Di trú động máy ảo (Live Migration), Tự động mở rộng tài nguyên (Auto-scaling CPU/RAM), và Tự phục hồi hệ thống khi sập node (Self-healing).
- Cải tiến 4 - Tích hợp Kho tri thức RAG: Ứng dụng Vector Database (ChromaDB) để lưu trữ tài liệu kỹ thuật vận hành và logs sự cố lịch sử, giúp AI Agent suy luận tìm nguyên nhân gốc rễ (Root Cause Analysis) chuẩn xác thay vì chỉ dựa vào

prompt thô.

Phương pháp thực hiện (Nêu tổng quan phương pháp thực hiện):

- Đề tài áp dụng kiến trúc hệ thống 4 lớp dựa trên khung điều khiển tự trị MAPE-K (Monitor - Analyze - Plan - Execute - Knowledge):

1. Lớp hạ tầng (Managed Element): Triển khai cụm phân tán (Cluster) gồm tối thiểu 3 node Raspberry Pi 4/5 chạy Ubuntu Server ARM64. OpenNebula Frontend đóng vai trò điều phối trung tâm.
2. Lớp giám sát và Tri thức (Monitor & Knowledge): Sử dụng Node Exporter và Prometheus để thu thập liên tục 5 chỉ số tài nguyên cốt lõi (CPU, RAM, Disk, Network, Load Average) sau mỗi 15 giây. Lưu trữ logs vào SQLite và ChromaDB (Knowledge).
3. Lớp xử lý AI (Analyze & Plan): Sử dụng framework CrewAI để điều phối nhóm Agent chuyên biệt (Monitor Agent, Analyst Agent, Remediation Architect). Các Agent sử dụng kỹ thuật Adapter để gọi mô hình Local LLM/SLM (thông qua Ollama).
4. Lớp thực thi và Giao diện (Execute): Xây dựng FastMCP Server đóng gói các công cụ CLI của OpenNebula (onevm, onehost) thành các hàm an toàn. Giao diện Web (FastAPI Backend + Vanilla JS/Tailwind CSS Frontend) đồng bộ dữ liệu thời gian thực cho phép quản trị viên giám sát và chuyển đổi linh hoạt giữa chế độ Manual (Human-in-the-loop) và Autonomous.

Các nội dung chính và giới hạn của đề tài (Nêu nội dung thực hiện và phương pháp dự kiến để đánh giá hệ thống, ví dụ, nếu KLTN có chạy demo thì hệ thống demo sẽ như thế nào):

- Các nội dung thực hiện chính:

- Nghiên cứu cài đặt và tối ưu hóa OpenNebula Node KVM trên phần cứng ARM64 của Raspberry Pi.
- Cấu hình hệ thống giám sát Prometheus và thiết lập Webhook đẩy cảnh báo tự động từ Alertmanager sang Backend API.
- Xây dựng và tối ưu hóa Local LLM chạy bằng Ollama trên Edge Node (hoặc PC cứu trợ mạng nội bộ).
- Lập trình hệ thống đa tác nhân CrewAI kết hợp FastMCP Server để điều khiển hạ tầng ảo hóa máy ảo.
- Nâng cấp ứng dụng Web quản trị (AIOps Dashboard, System Log Stream, AI Chatbox tương tác hạ tầng).
- Áp dụng quy trình GitOps để quản lý và tự động hóa triển khai toàn bộ cấu hình hệ thống từ kho lưu trữ Git.

- Giới hạn của đề tài:
- Hệ thống thử nghiệm ở quy mô mô phỏng phòng thí nghiệm (Cụm biên phân tán nhỏ từ 3 đến 5 node Raspberry Pi).
 - Mô hình Local LLM giới hạn ở các dòng mô hình nhỏ có kích thước dưới 8 tỷ tham số ($\leq 8B$ parameters) để đảm bảo tốc độ suy luận ở biên.
 - Hệ điều hành chạy trong các máy ảo (VMs) trên cụm Pi giới hạn ở các bản phân phối Linux siêu nhẹ (như Alpine Linux hoặc Ubuntu Core ARM64) để phù hợp với dung lượng RAM của thiết bị nhúng.
- Phương pháp dự kiến để đánh giá hệ thống (Kịch bản chạy Demo): Hệ thống sẽ chạy thử nghiệm demo trực tiếp trước Hội đồng tốt nghiệp với mô hình vật lý cụ thể thông qua 2 kịch bản tải nặng:
- Kịch bản Demo 1 - Tự động di trú khắc phục quá tải (Live Migration): Chạy công cụ stress-ng ép CPU/RAM trên một máy ảo tại Node Pi số 2 lên $> 90\%$. Biểu đồ Grafana trên Web App lập tức chuyển sang trạng thái đỏ (CRITICAL). Alertmanager kích hoạt AI Agent. AI Agent tự động chẩn đoán lỗi, lập kế hoạch và thực thi lệnh Live Migration di dời máy ảo bị quá tải đó sang Node Pi số 3 (còn trống tài nguyên) hoàn toàn tự động. Chứng minh dịch vụ Web trên máy ảo không bị gián đoạn và đồ thị tải tại Node Pi số 2 giảm về mức an toàn dưới 2 phút.
 - Kịch bản Demo 2 - Tự phục hồi hệ thống (Self-healing & Scaling): Giả lập lỗi tắt đột ngột (rút nguồn vật lý) của một Node Pi đang chạy máy ảo quan trọng. AI Agent dựa trên dữ liệu snapshot cuối cùng và Kho tri thức RAG sẽ tự động phát hiện node sập, ra lệnh khởi tạo lại (provisioning) một máy ảo thay thế trên các Node Pi còn sống để khôi phục dịch vụ của hệ thống biên.

Kế hoạch thực hiện:*(Mô tả kế hoạch làm việc và phân công công việc cho từng sinh viên tham gia)*

Tuần	Nội dung công việc	Sinh viên thực hiện	Kết quả cần đạt (Deliverables)
1 - 2	Nghiên cứu tài liệu sâu về ảo hóa ARM64, Local LLM (Ollama), và giao thức MCP Server mở rộng.	Cả 2 sinh viên	Báo cáo đề cương chi tiết và sơ đồ kiến trúc hệ thống tổng thể.
3 - 4	Triển khai hạ tầng vật lý cụm Raspberry Pi Cluster. Cài đặt hệ điều hành Ubuntu Server ARM64 và cấu hình cluster OpenNebula (Frontend + KVM Nodes).	Lê Xuân Hoàng	Cụm Pi Cluster chạy ổn định, tạo và quản lý được máy ảo Alpine Linux thủ công qua Sunstone.

3 - 4	Cài đặt và cấu hình Node Exporter, Prometheus trung tâm, Alertmanager và thiết kế Grafana Dashboard chuyên sâu.	Đặng Minh Dzũ	Hệ thống thu thập metrics thời gian thực, nhúng thành công các iframe panel Grafana.
5 - 7	Thiết lập môi trường Local AI với Ollama (Llama-3/Qwen-2.5). Lập trình hệ thống đa tác nhân CrewAI cho 3 Agent (Monitor, Analyze, Plan).	Đặng Minh Dzũ	AI Agent suy luận chạy offline hoàn toàn, xuất ra kế hoạch sửa lỗi định dạng JSON chuẩn.
5 - 7	Lập trình phát triển bộ công cụ mở rộng trên FastMCP Server kết nối trực tiếp với OpenNebula API (các hàm migrate, scale, reboot).	Lê Xuân Hoàng	MCP Server phơi bày các hàm quản trị hệ thống dưới dạng Function Calling an toàn.
8 - 9	Xây dựng cơ sở dữ liệu tri thức SQLite kết hợp Vector Database ChromaDB, lập trình luồng RAG nạp tri thức vận hành hệ thống.	Đặng Minh Dzũ	AI Agent có khả năng tra cứu logs lịch sử và tài liệu kỹ thuật để tối ưu hóa quyết định.
8 - 9	Phát triển ứng dụng Web quản trị: Viết FastAPI Backend và nâng cấp giao diện Frontend Single Page (Real-time logs stream, chatbox tương tác hạ tầng).	Lê Xuân Hoàng	Giao diện Web App tương tác mượt mà, đồng bộ dữ liệu bằng Polling 2 giây/lần.
10 - 11	Tích hợp toàn diện hệ thống: Kết nối Web App - CrewAI Agent - Local LLM - MCP Server - Hạ tầng OpenNebula Pi Cluster. Cấu hình GitOps pipeline.	Cả 2 sinh viên	Hệ thống vận hành khép kín chu trình MAPE-K từ đầu đến cuối.
12 - 13	Chạy các kịch bản kiểm thử giả lập sự cố (quá tải CPU, cạn kiệt RAM, rút nguồn sập node). Thu thập dữ liệu định lượng và tinh chỉnh prompt.	Cả 2 sinh viên	Hệ thống vượt qua các tiêu chí kiểm thử (độ trễ phản hồi < 2 phút, độ chính xác > 85%).
14 - 15	Hoàn thiện quyền báo cáo Khóa luận tốt nghiệp, chuẩn bị Slide thuyết trình và mô hình phần cứng để mang lên hội đồng demo.	Cả 2 sinh viên	Quyển KLTN hoàn chỉnh và hệ thống mô hình sẵn sàng bảo vệ.
Xác nhận của CBHD (Ký tên và ghi rõ họ tên)		TP. HCM, ngày....thángnăm..... Sinh viên (Ký tên và ghi rõ họ tên)	